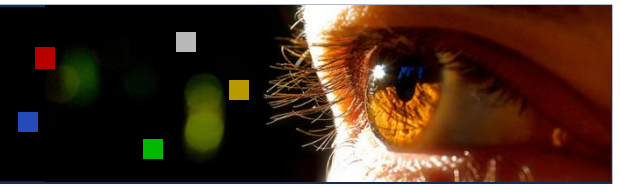
Visión por Computador

Versión 2026

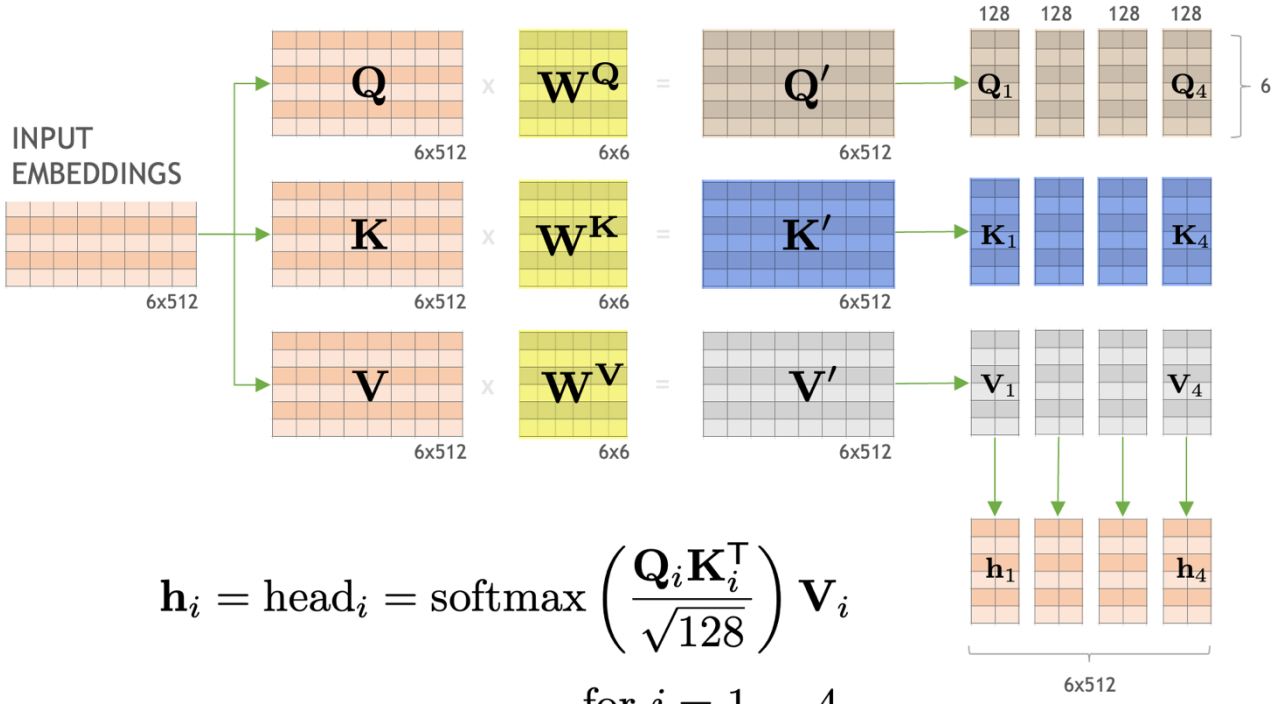


Transformers

(CAPÍTULO 3 – CV03F)

Domingo Mery

Ciencia de la Computación
Universidad Católica – Chile
http://domingomery.ing.uc.cl



$$\mathbf{h}_i = \text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i\mathbf{K}_i^\mathsf{T}}{\sqrt{128}}\right)\mathbf{V}_i$$

$$\text{for } i = 1 \ldots 4$$

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

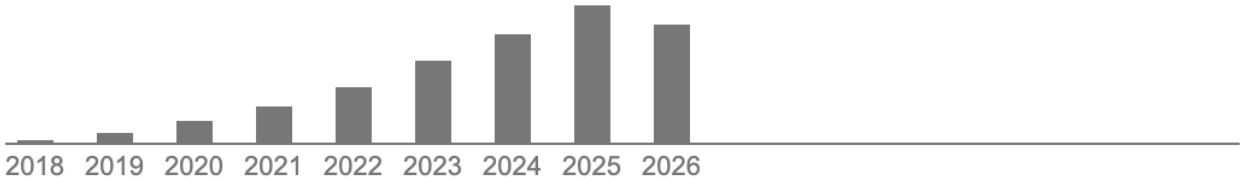
Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).



Ashish Vaswani

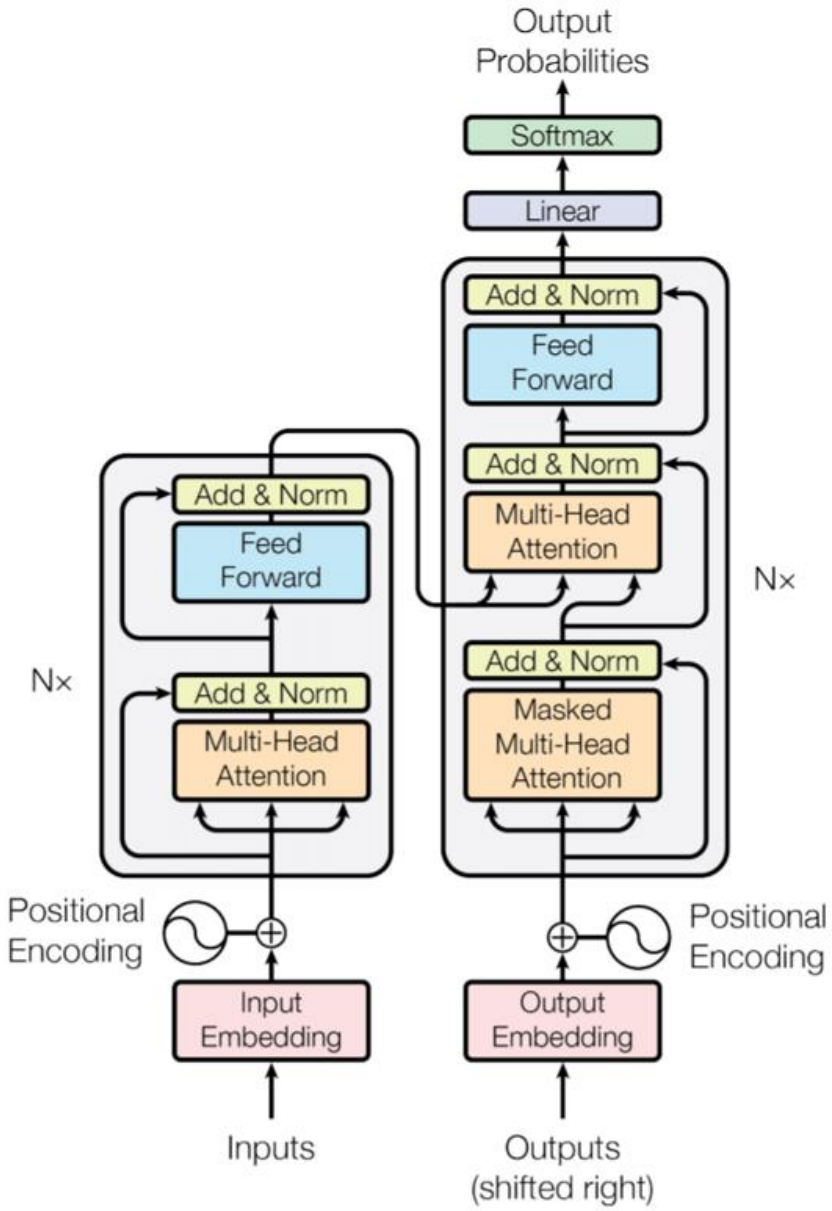
Attention is all you need

Authors	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, Illia Polosukhin
Publication date	2017
Journal	Advances in neural information processing systems
Volume	30
Description	The dominant sequence transduction models are based on complex recurrent orconvolutional neural networks in an encoder and decoder configuration. The best performing such models also connect the encoder and decoder through an attentionm echanisms. We propose a novel, simple network architecture based solely onan attention mechanism, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superiorin quality while being more parallelizable and requiring significantly less timeto train. Our single model with 165 million parameters, achieves 27.5 BLEU onEnglish-to-German translation, improving over the existing best ensemble result by over 1 BLEU. On English-to-French translation, we outperform the previoussingle state-of-the-art with model by 0.7 BLEU, achieving a BLEU score of 41.1.
Total citations	Cited by 294539



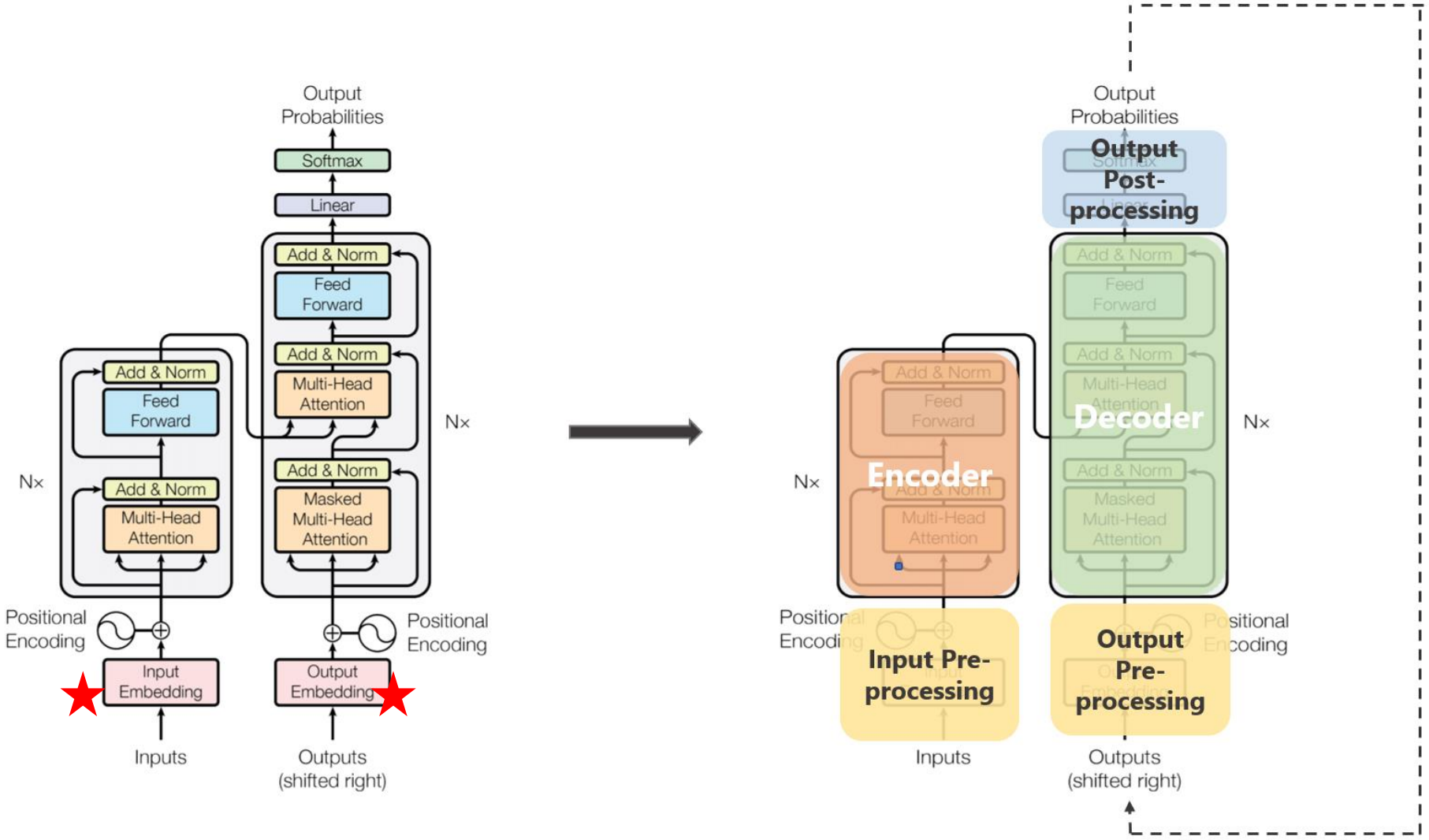
Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).

Transformers



Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." Advances in neural information processing systems 30 (2017).

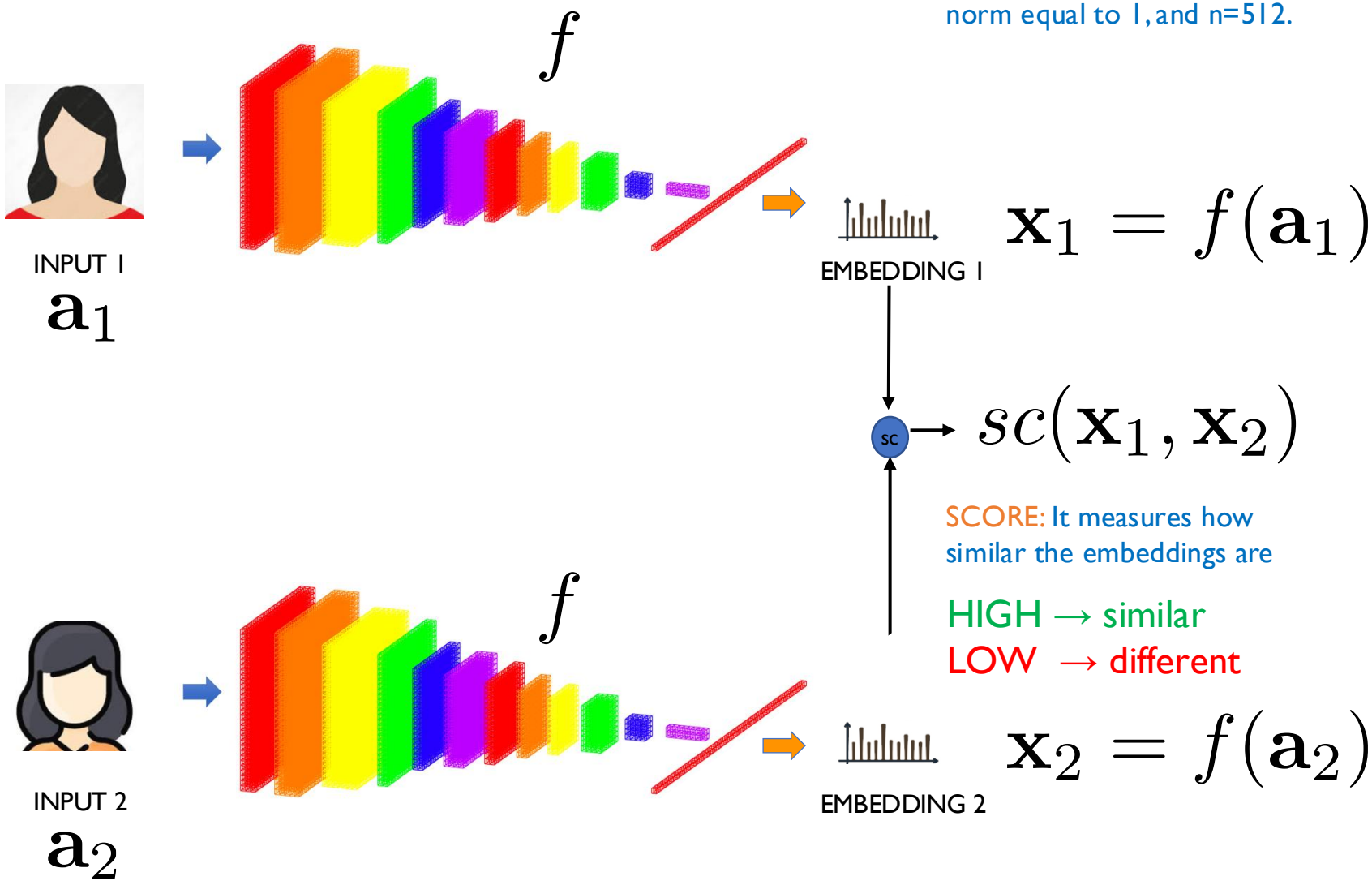
Transformers



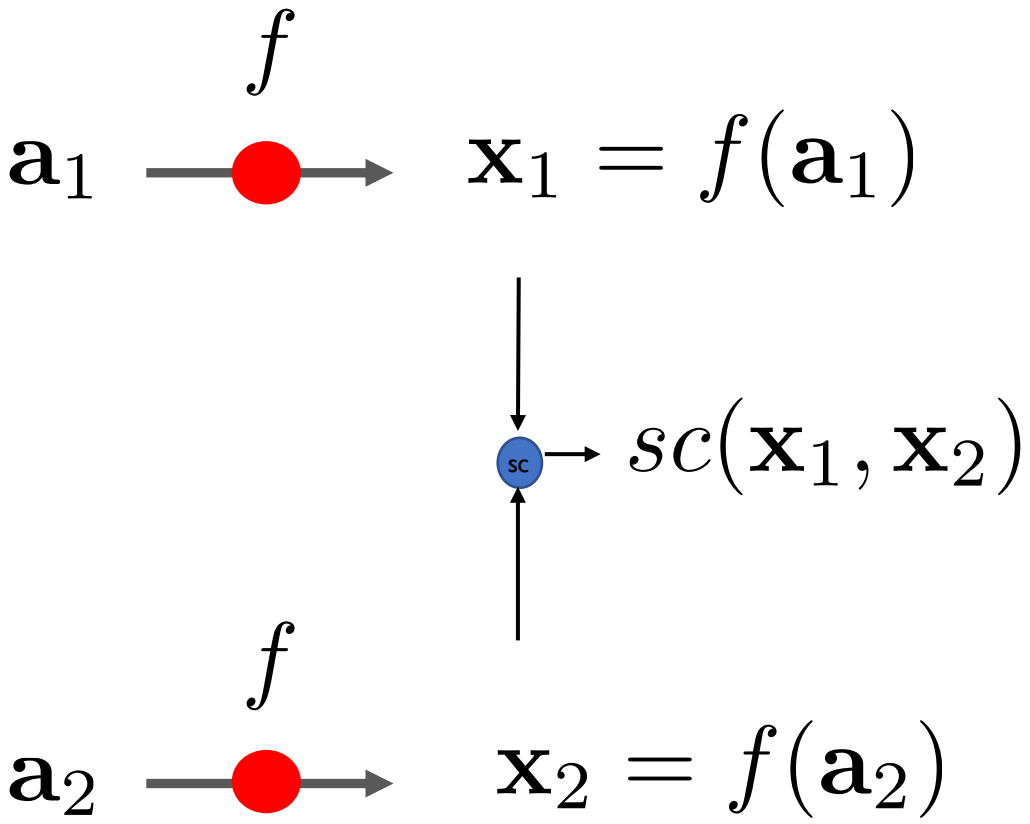
From https://towardsdatascience.com/transformers-89034557de14

Embeddings

EMBEDDINGS: vectors of n elements that describe the input. Typically, they have a norm equal to 1, and n=512.



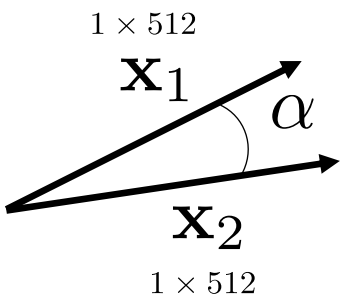
Embeddings


f
$\mathbf{a}_1$
$\mathbf{x}_1 = f(\mathbf{a}_1)$
sc
$sc(\mathbf{x}_1, \mathbf{x}_2)$
f
$\mathbf{a}_2$
$\mathbf{x}_2 = f(\mathbf{a}_2)$

Embeddings

HOW SIMILAR ARE TWO VECTORS?

DON'T FORGET: in our case the vectors have 512 elements and have a norm equal to 1



DOT PRODUCT

$$\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = ||\mathbf{x}_1|| ||\mathbf{x}_2|| \cos(\alpha)$$

$$\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = \cos(\alpha) \quad \text{SCORE}$$

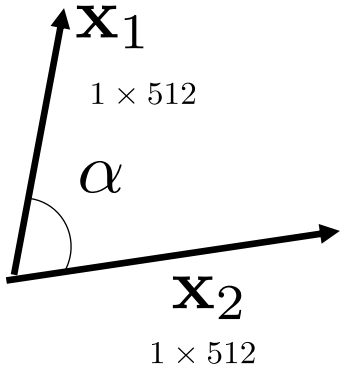
$$\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = \mathbf{x}_1 \mathbf{x}_2^T$$

- If the vectors are very similar, the angle between them is small and the SCORE is close to 1.

Embeddings

HOW SIMILAR ARE TWO VECTORS?

DON'T FORGET: in our case the vectors have 512 elements and have a norm equal to 1



DOT PRODUCT

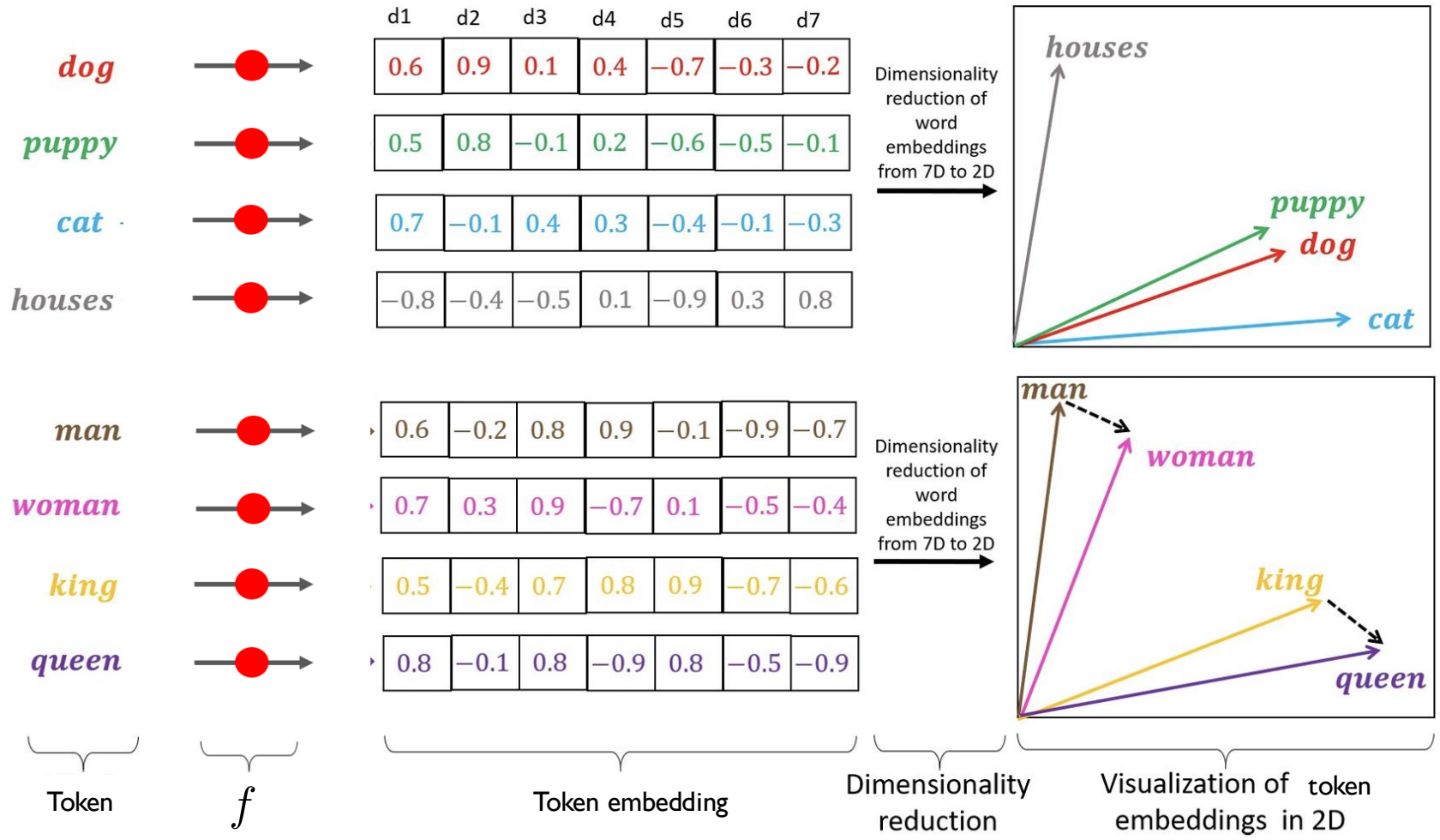
$$\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = ||\mathbf{x}_1|| ||\mathbf{x}_2|| \cos(\alpha)$$

$$\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = \cos(\alpha) \quad \text{SCORE}$$

$$\langle \mathbf{x}_1, \mathbf{x}_2 \rangle = \mathbf{x}_1 \mathbf{x}_2^T = sc(\mathbf{x}_1, \mathbf{x}_2)$$

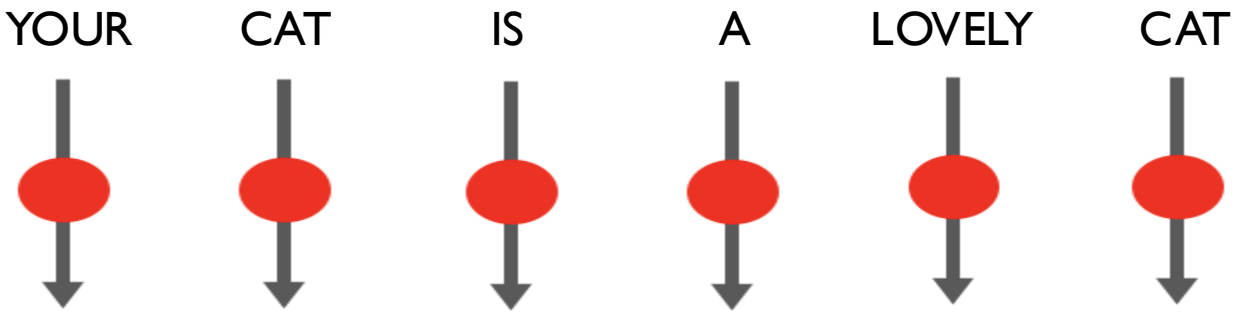
- If the vectors are very similar, the angle between them is small and the SCORE is close to 1.
- If the vectors are very different, the angle between them is large and the SCORE decreases significantly.

Embeddings in NLP



Adapted from: Rozado, D. (2020). Wide range screening of algorithmic bias in word embedding models using large sentiment lexicons reveals underreported bias types. PloS one, 15(4), e0231189.

Embeddings in NLP

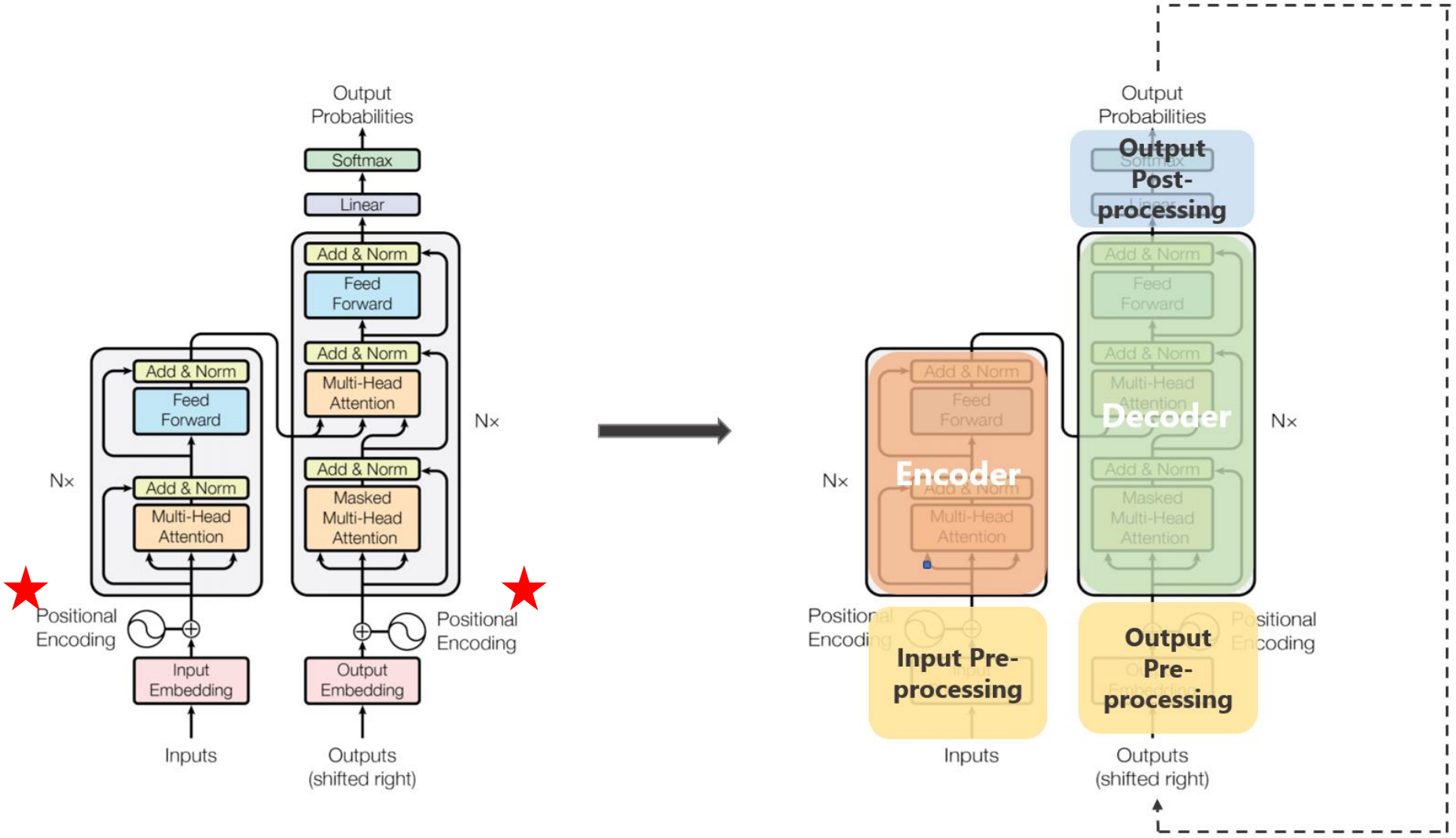


0.6498	0.1423	0.6575	0.5058	0.5041	0.1423
0.0488	0.4471	0.9104	0.7922	0.3331	0.4471
0.1866	0.7632	0.1610	0.2960	0.6215	0.7632
0.3649	0.4053	0.3665	0.5180	0.2484	0.4053
0.5233	0.9723	0.3637	0.3431	0.5979	0.9723
0.1795	0.8279	0.8956	0.0108	0.8130	0.8279
0.3982	0.8096	0.1930	0.7688	0.6815	0.8096
0.6069	0.3642	0.0910	0.1931	0.2591	0.3642
0.7212	0.4306	0.9693	0.1644	0.3752	0.4306
:	:	:	:	:	:
0.8523	0.9218	0.1417	0.6043	0.8187	0.9218

Embeddings de las palabras

Adapted from: Umar Jamil: Attention is all you need (Transformer) – Model explanation.
https://github.com/hkproj/transformer-from-scratch-notes

Transformers



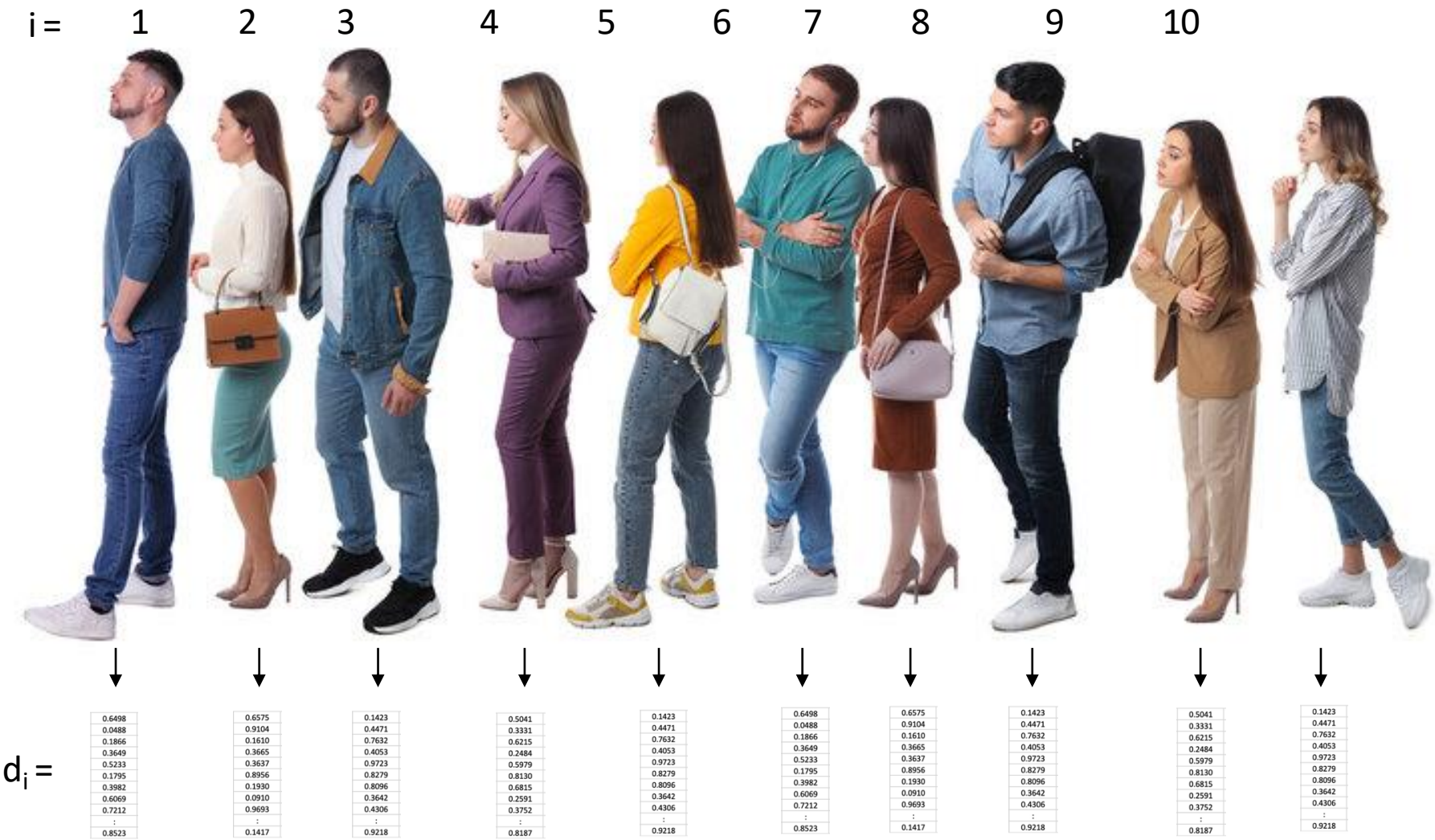
From https://towardsdatascience.com/transformers-89034557de14

Positional Encoding (PE)



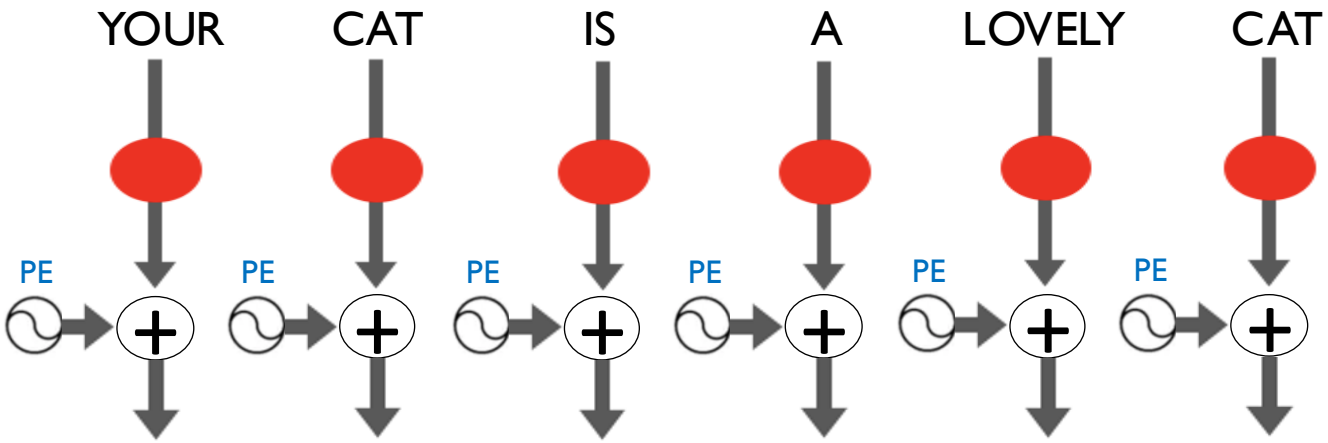
$$||\mathbf{d}_j - \mathbf{d}_i|| < ||\mathbf{d}_k - \mathbf{d}_i|| \quad \text{if} \quad k > j > i \quad \text{for persons} \quad i, j, k$$

Positional Encoding (PE)



$$||\mathbf{d}_j - \mathbf{d}_i|| < ||\mathbf{d}_k - \mathbf{d}_i|| \quad \text{if} \quad k > j > i \quad \text{for persons} \quad i, j, k$$

Positional Encoding (PE)



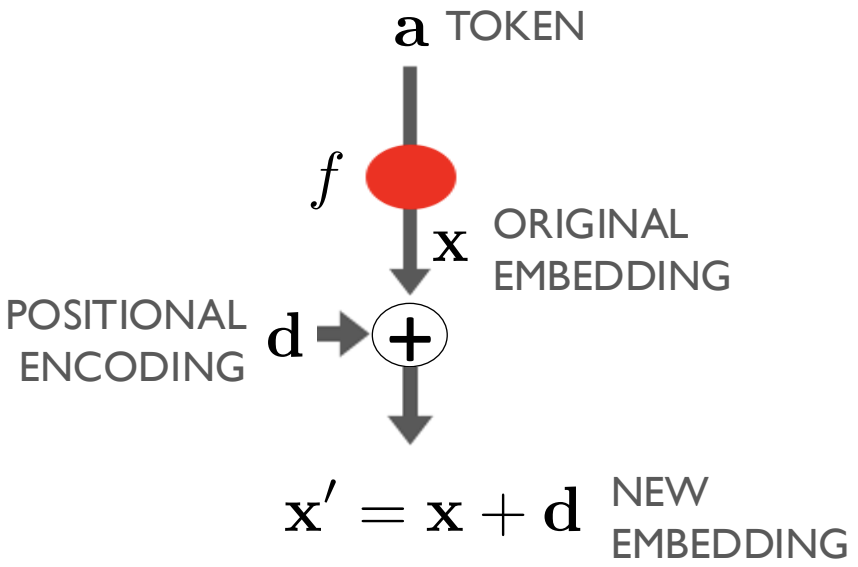
0.2791	0.7828	0.5191	0.8630	0.9760	0.7697
0.9383	0.9076	0.1494	0.9702	0.3678	0.5559
0.2092	0.4826	0.8694	0.6183	0.2479	0.3256
0.6192	0.2920	0.6926	0.4538	0.0441	0.7436
0.2390	0.3588	0.2785	0.3756	0.2979	0.7988
0.3657	0.3488	0.6655	0.7654	0.8425	0.8582
0.3218	0.0450	0.9454	0.3925	0.1291	0.7702
0.6912	0.5924	0.7275	0.4673	0.6499	0.2473
0.9746	0.2585	0.2245	0.9646	0.1241	0.7357
:	:	:	:	:	:
0.9410	0.0930	0.0351	0.1137	0.9607	0.3678

Embeddings de las palabras

+

Embeddings de las posiciones

Positional Encoding



- Each token has information about the position in the sentence.
- Tokens that are close to each other should be close in the embedding's space
- Tokens that are distant/different should be distant in the embedding's space

$$||\mathbf{d}_j - \mathbf{d}_i|| < ||\mathbf{d}_k - \mathbf{d}_i|| \quad \text{if} \quad k > j > i \quad \text{for persons} \quad i, j, k$$



Positional Encoding

For i even

$$PE(p,i) = \sin(p/10000^{i/d})$$

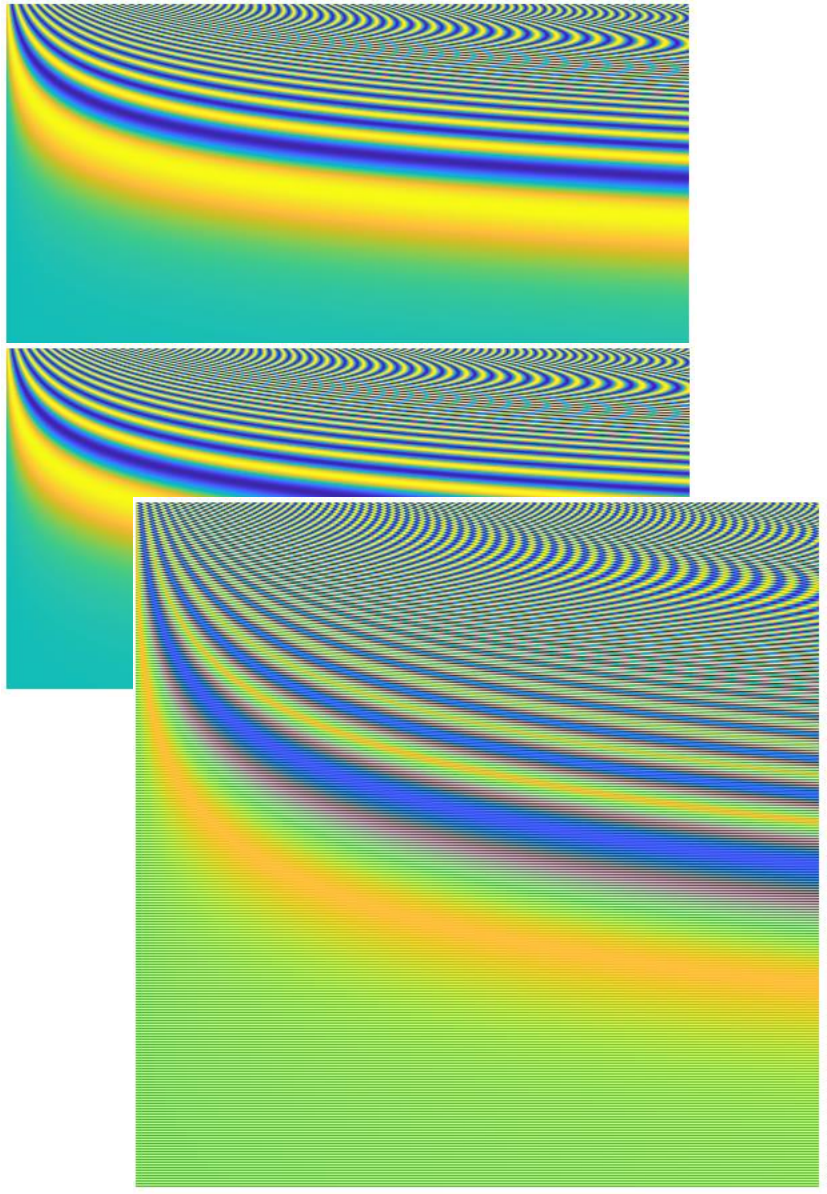
For i odd

$$PE(p,i) = \cos(p/10000^{(i-1)/d})$$

i: element index of the embedding

p: token position

d: embedding size (512 for example)



Positional Encoding

For i even

$$PE(p,i) = \sin(p/10000^{i/d})$$

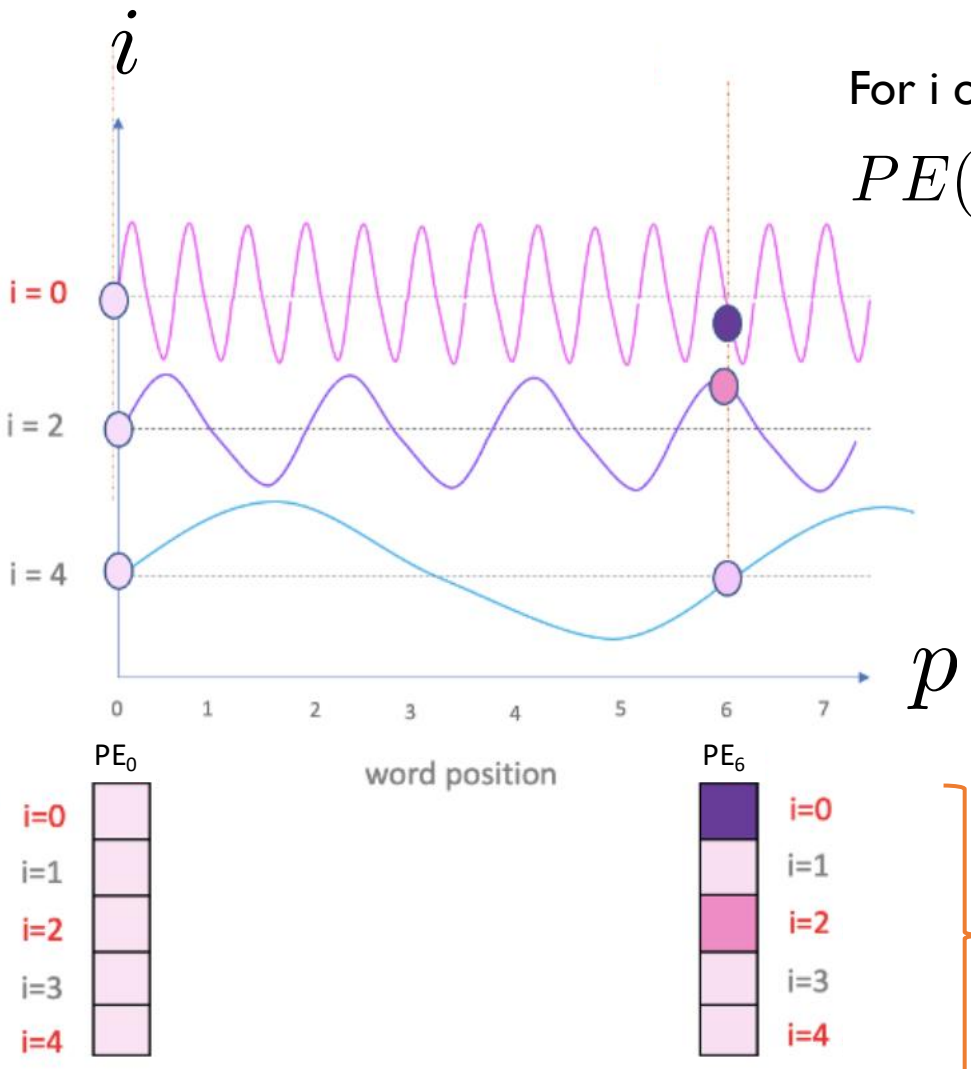
For i odd

$$PE(p,i) = \cos(p/10000^{(i-1)/d})$$

i: element index of the embedding

p: token position

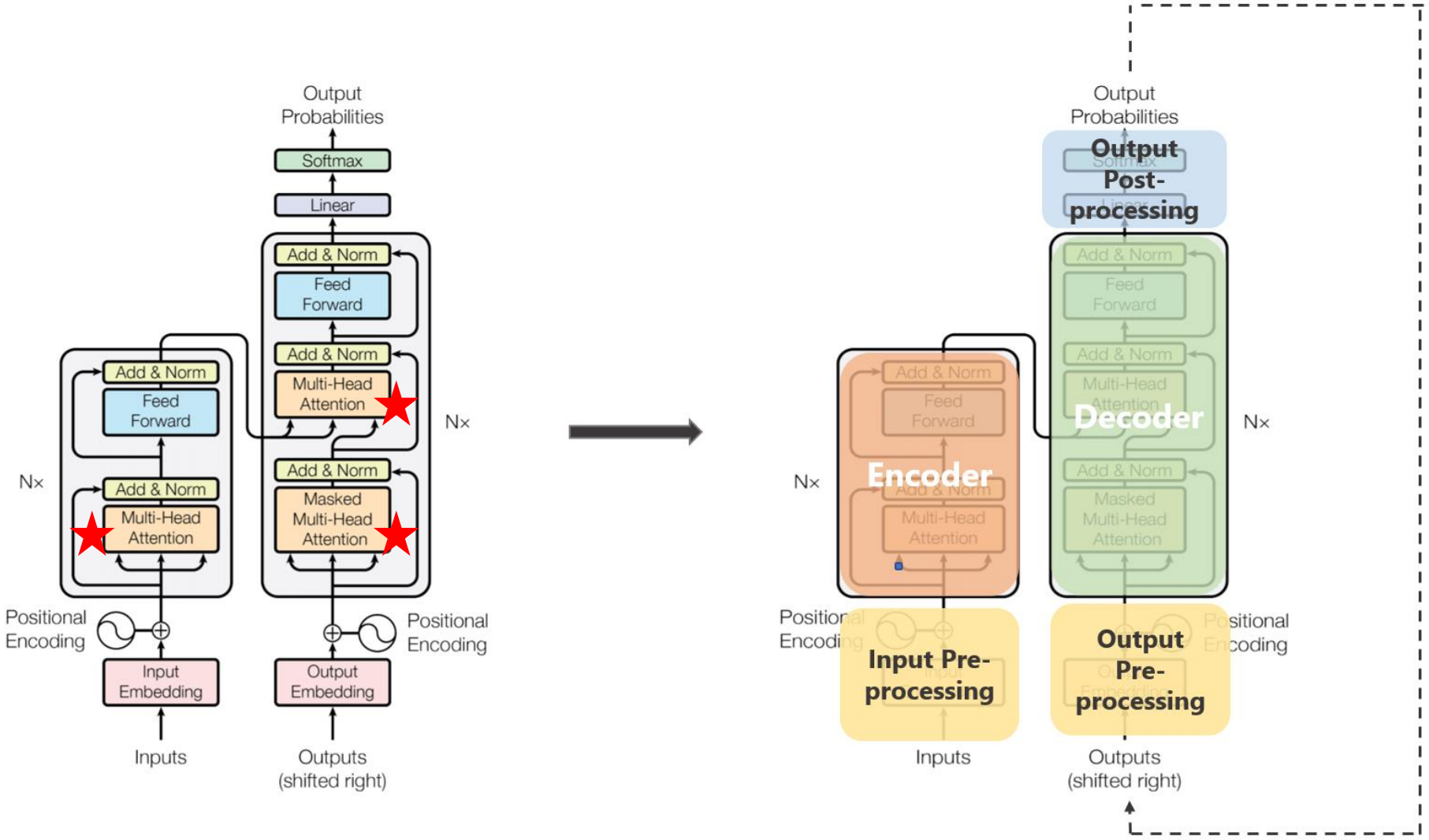
d: embedding size (512 for example)



Positional Encoding

Adapted from: https://datascience.stackexchange.com/questions/51065/what-is-the-positional-encoding-in-the-transformer-model

Transformers

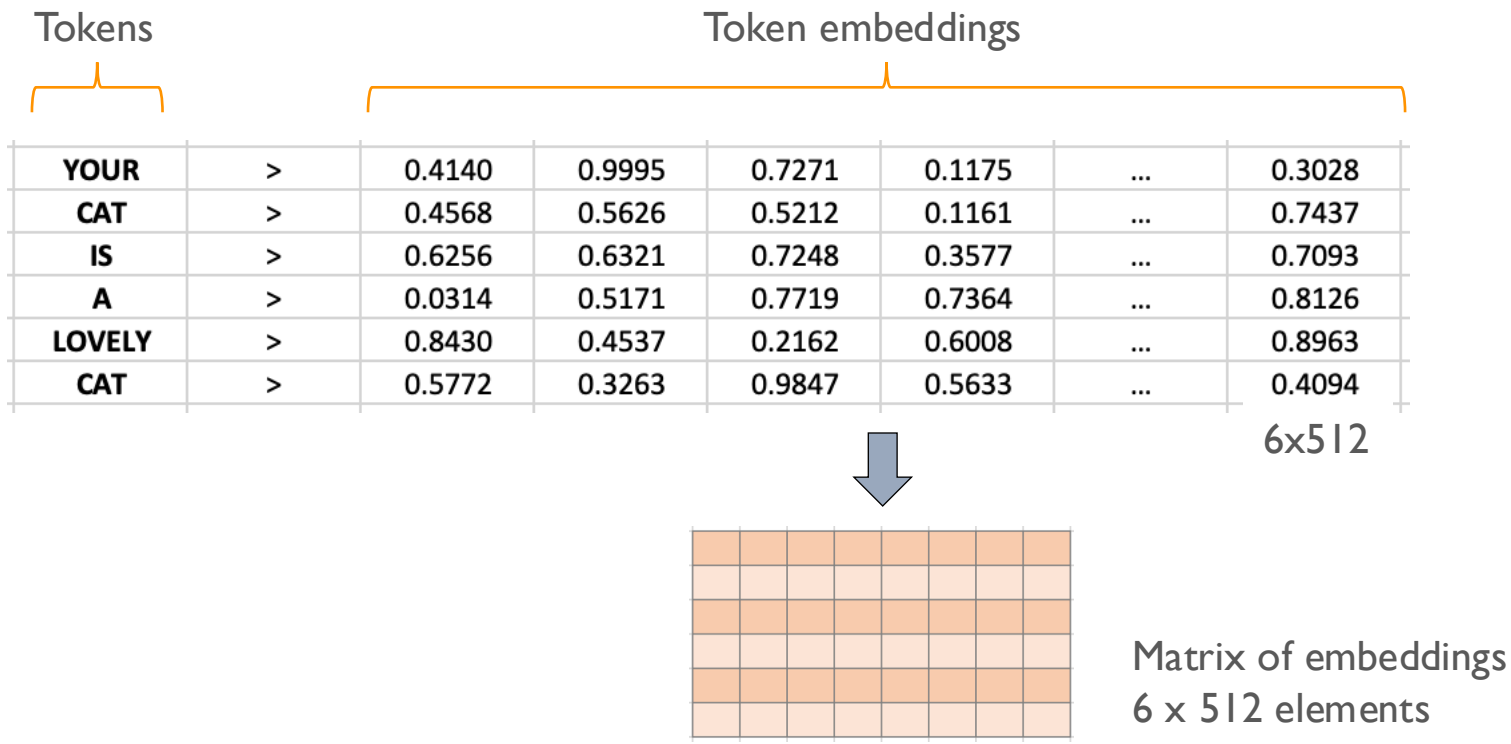


From https://towardsdatascience.com/transformers-89034557de14

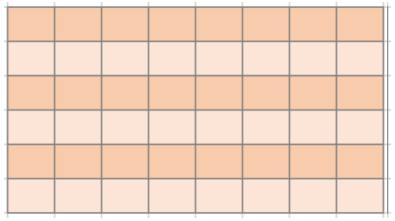
Self-Attention

It allows the model to relate tokens to each other

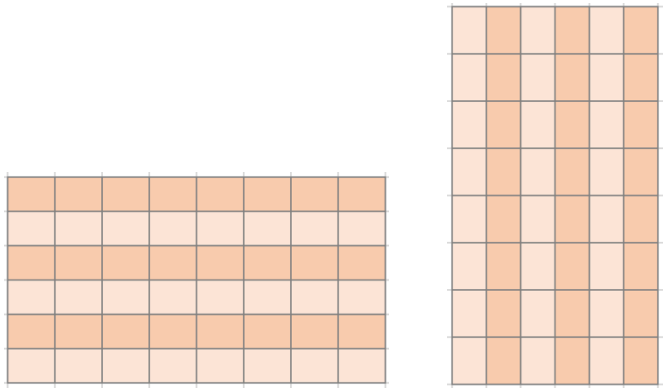
For 6 tokens with embeddings of 512 elements:



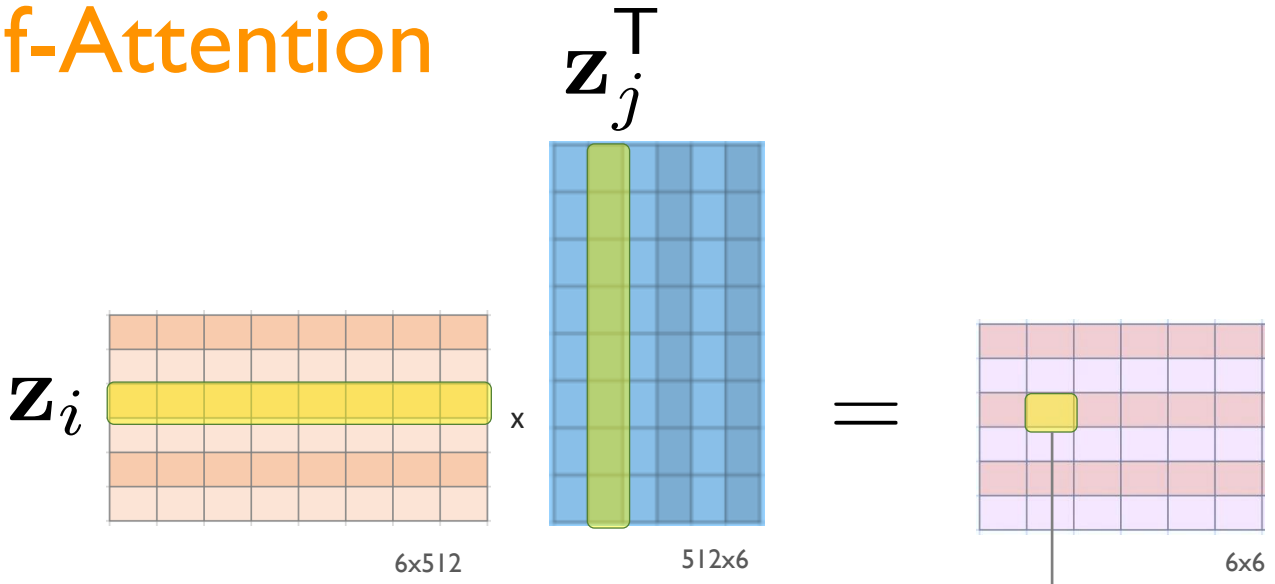
Self-Attention



Self-Attention



Self-Attention



$$z_{i,j} = \mathbf{z}_i \mathbf{z}_j^T$$

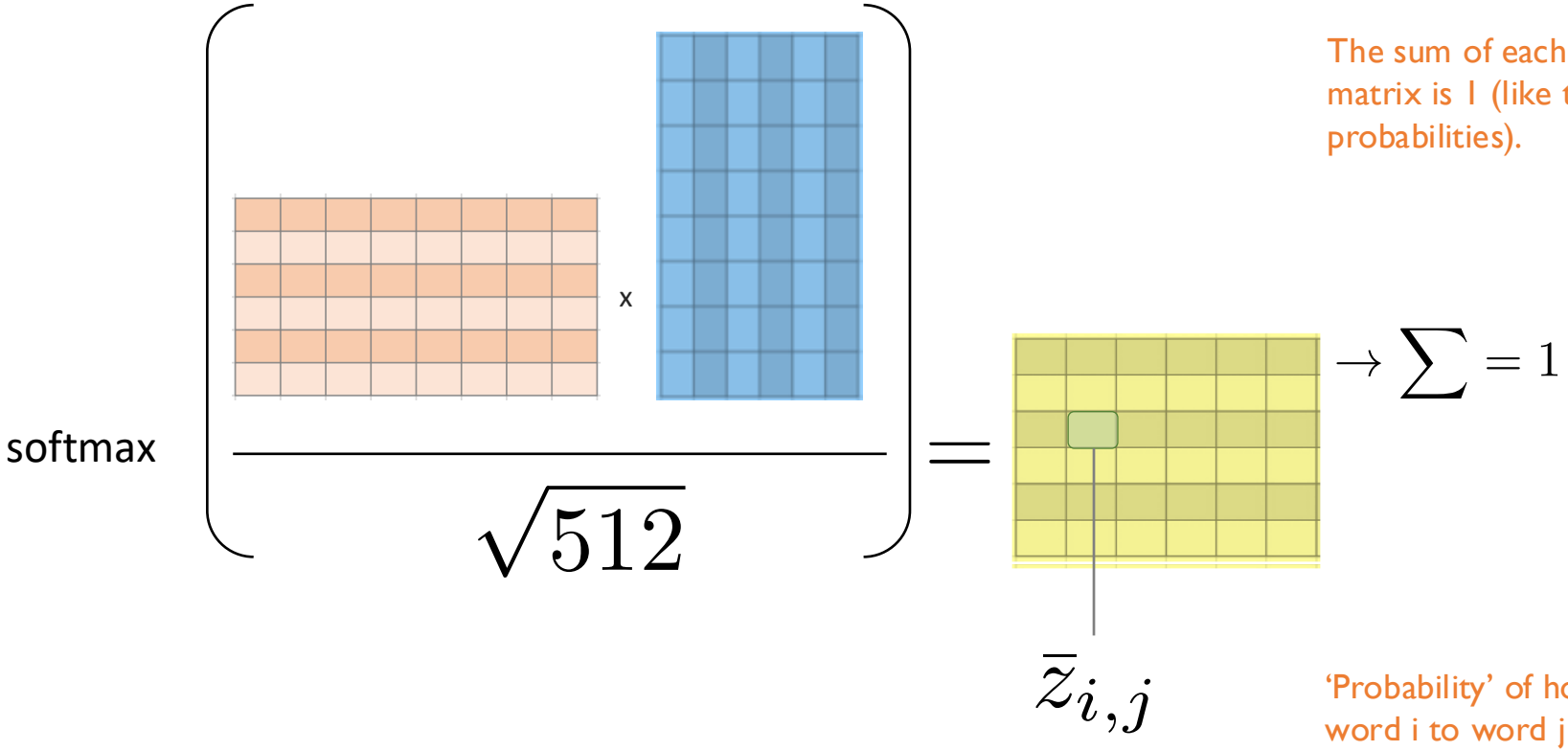
REMEMBER!

Element (i,j) means how similar is the embedding of token i with the embedding of token j

SCORE: It measures how similar the embeddings are

HIGH → similar
LOW → different

Self-Attention

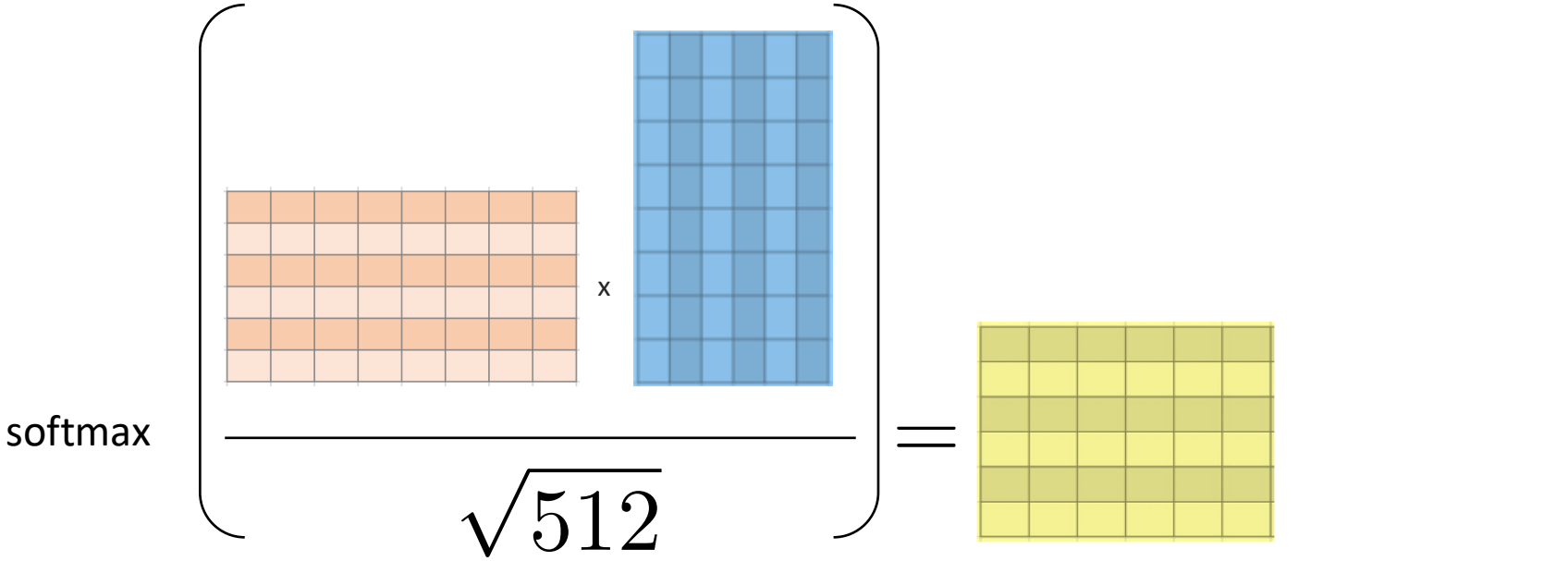


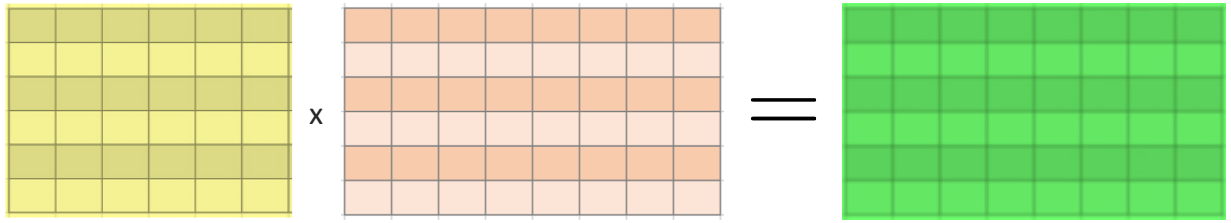
The sum of each row of the matrix is 1 (like the probabilities).

'Probability' of how similar is word i to word j (related to all words)

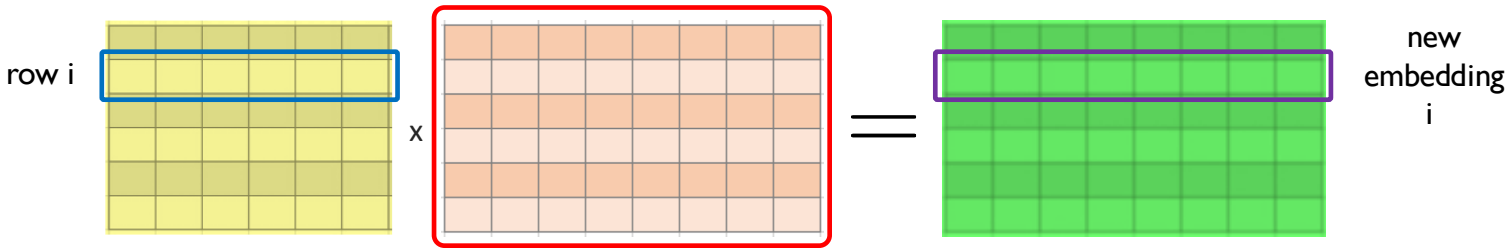
$$\bar{z}_{i,j} = \frac{e^{z_{i,j}}}{\sum_k e^{z_{i,k}}}$$

Self-Attention

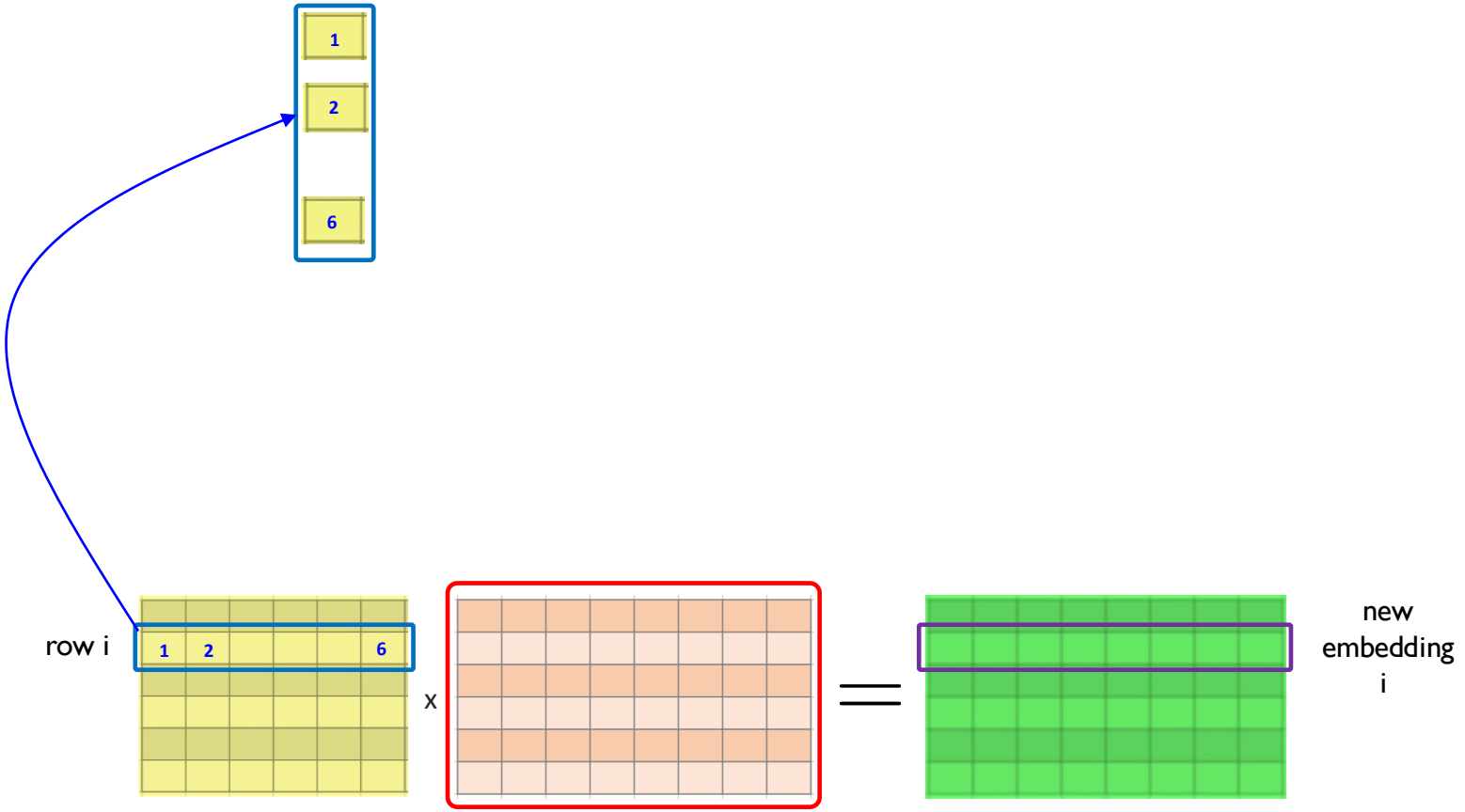




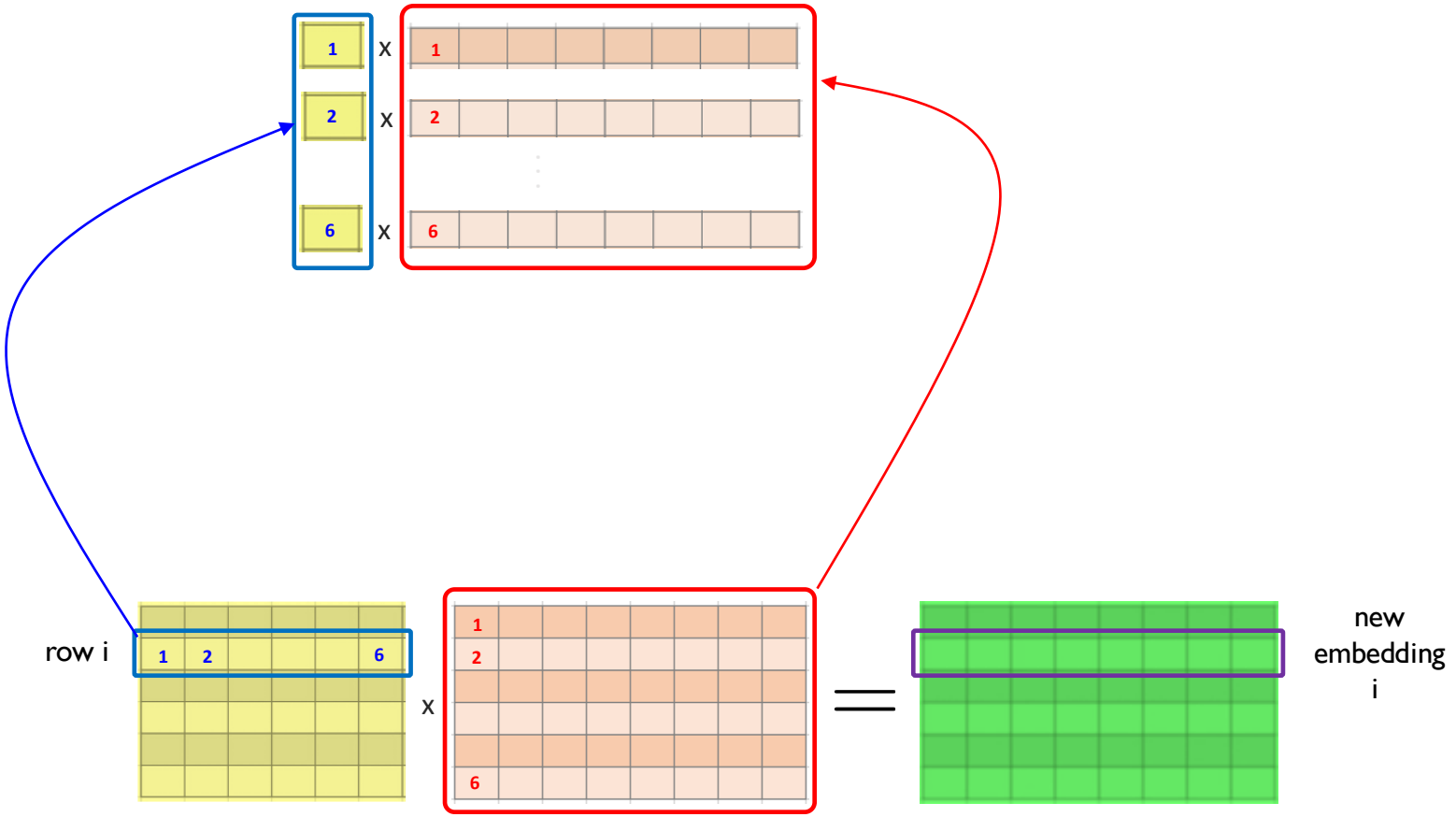
Self-Attention



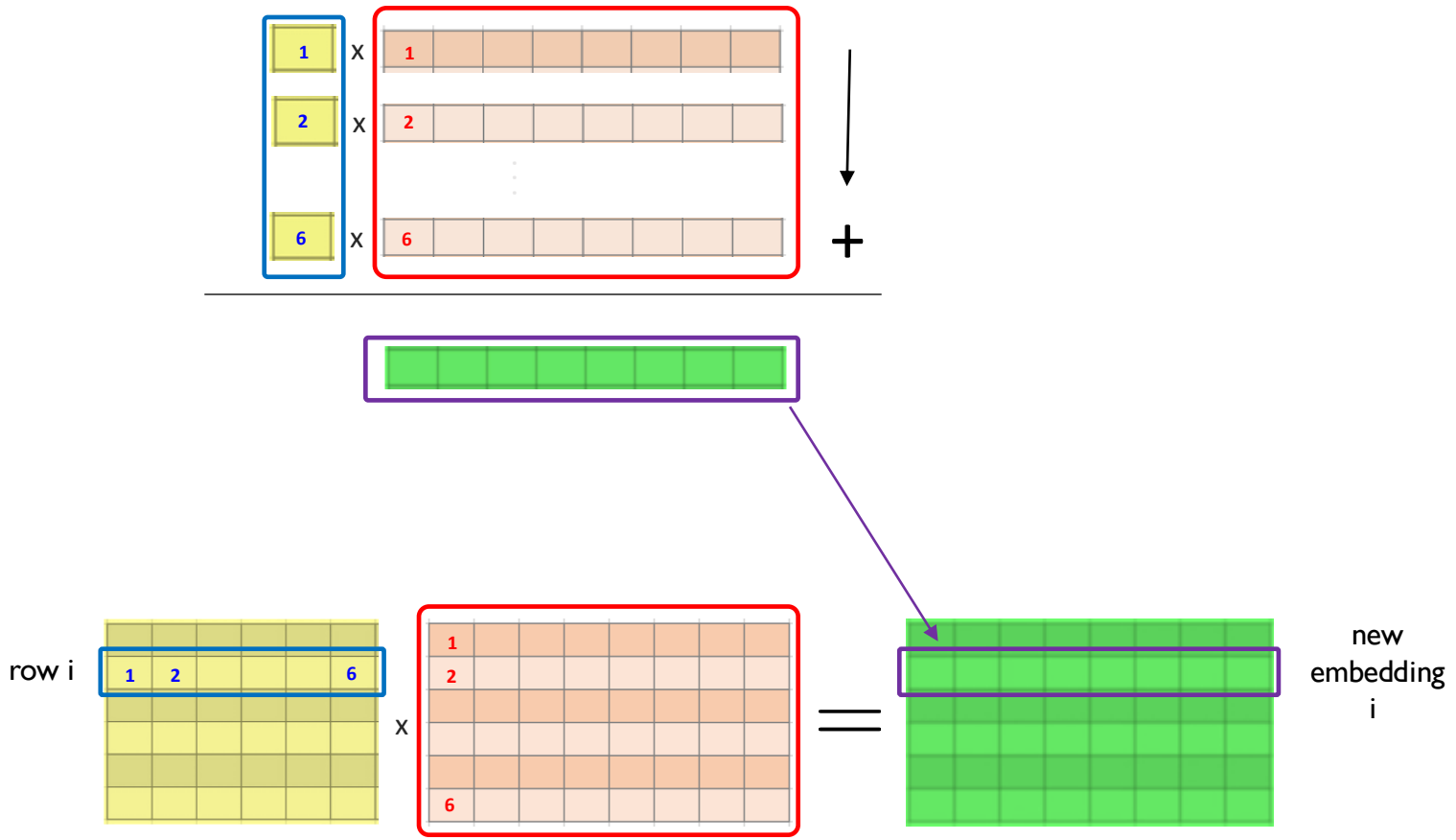
Self-Attention



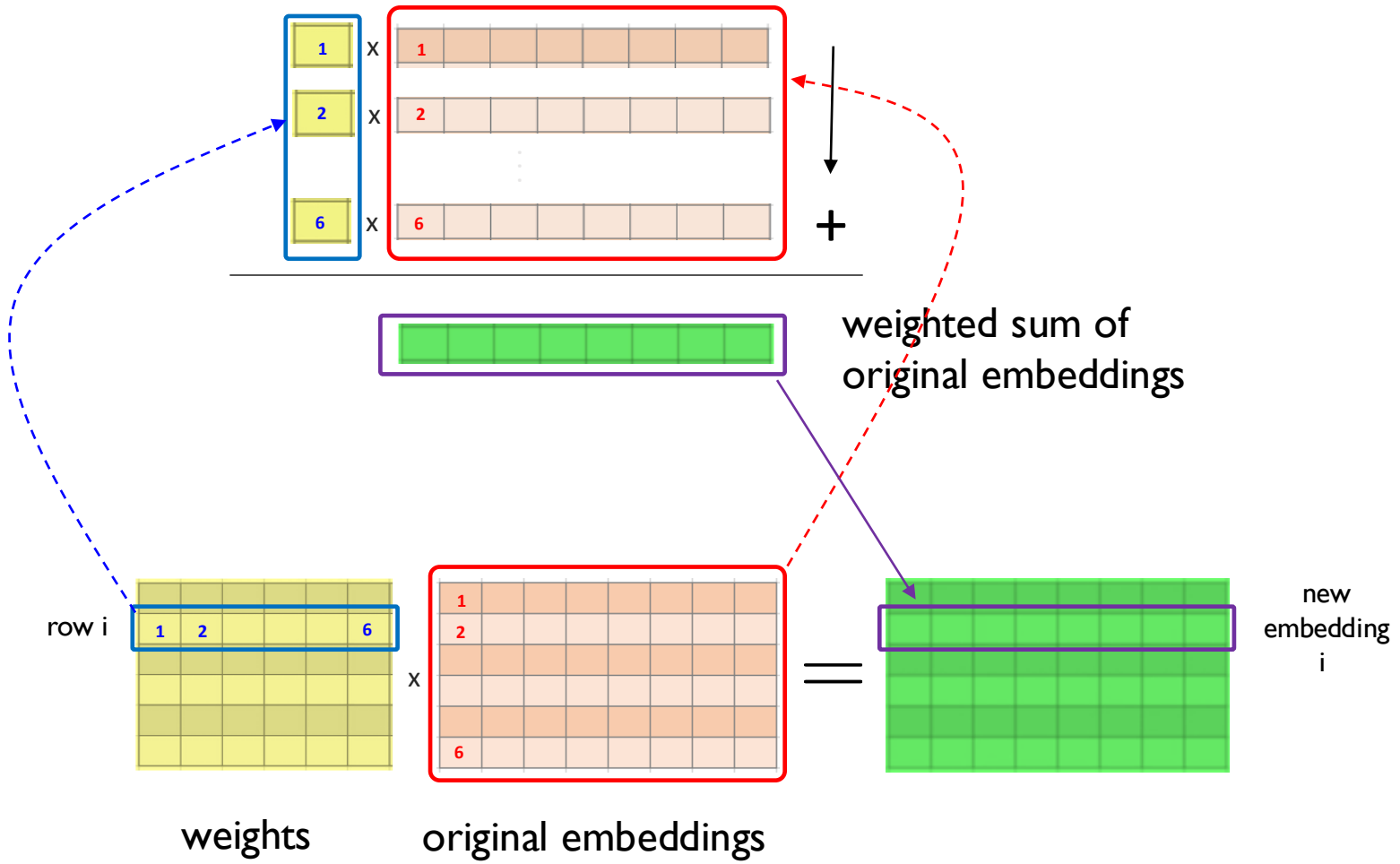
Self-Attention



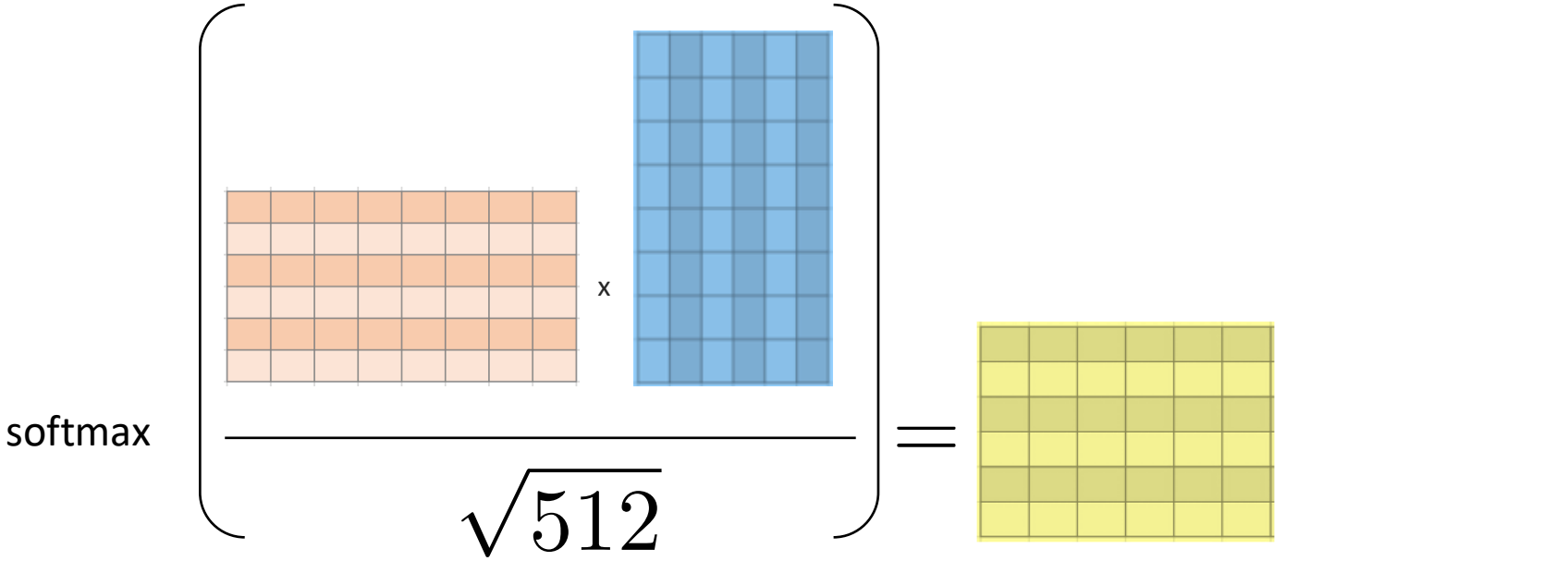
Self-Attention

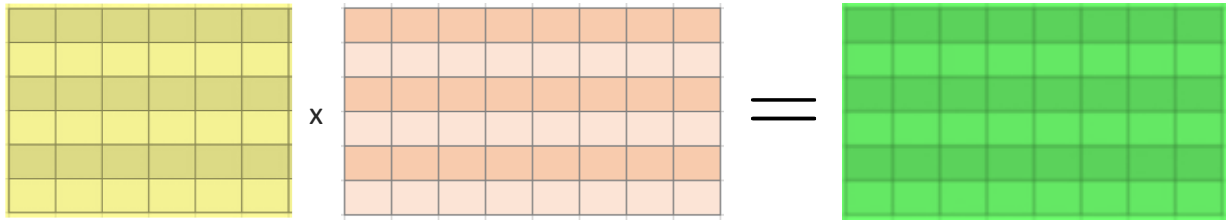


Self-Attention

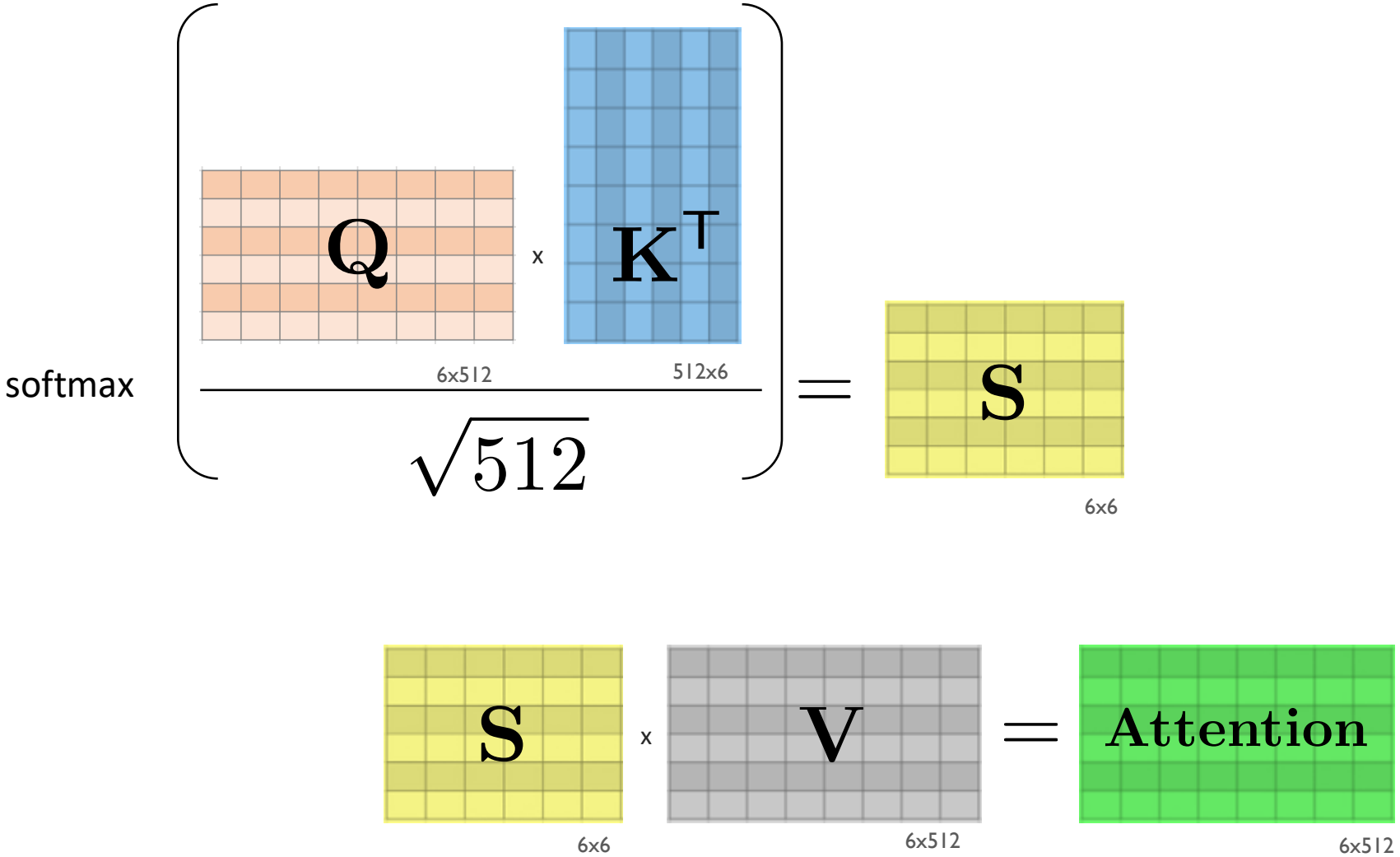


Self-Attention

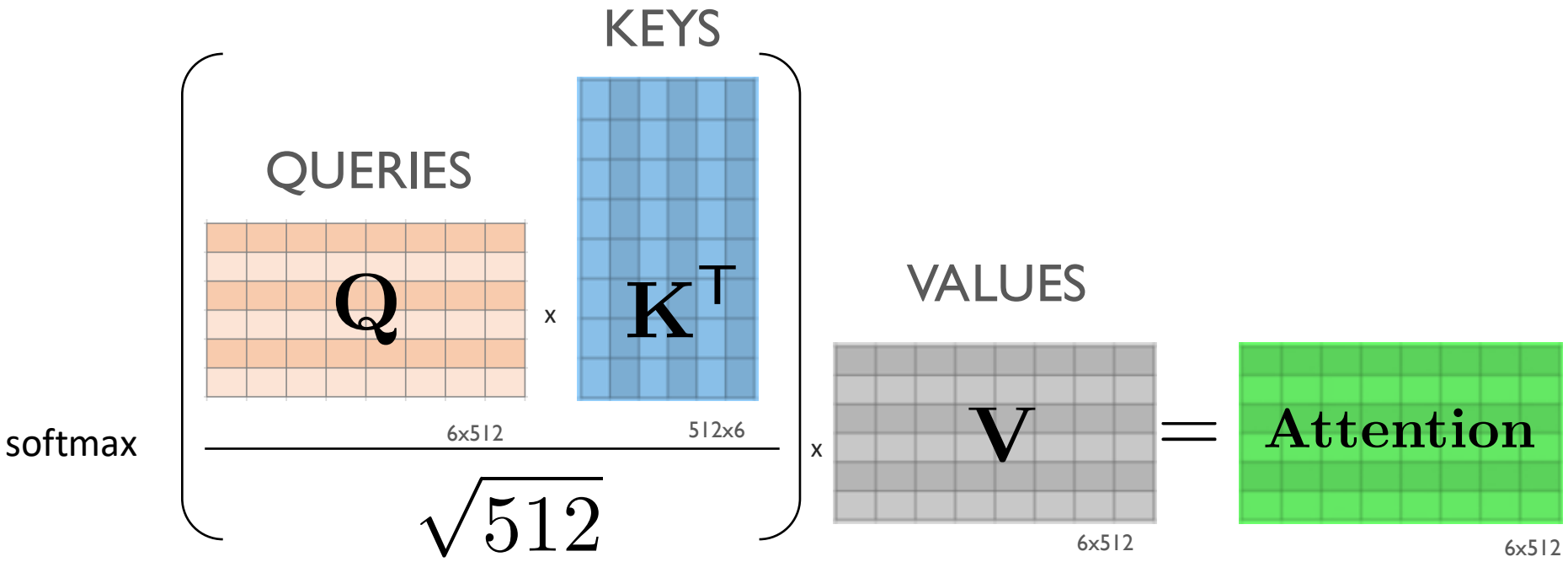




Self-Attention



Self-Attention

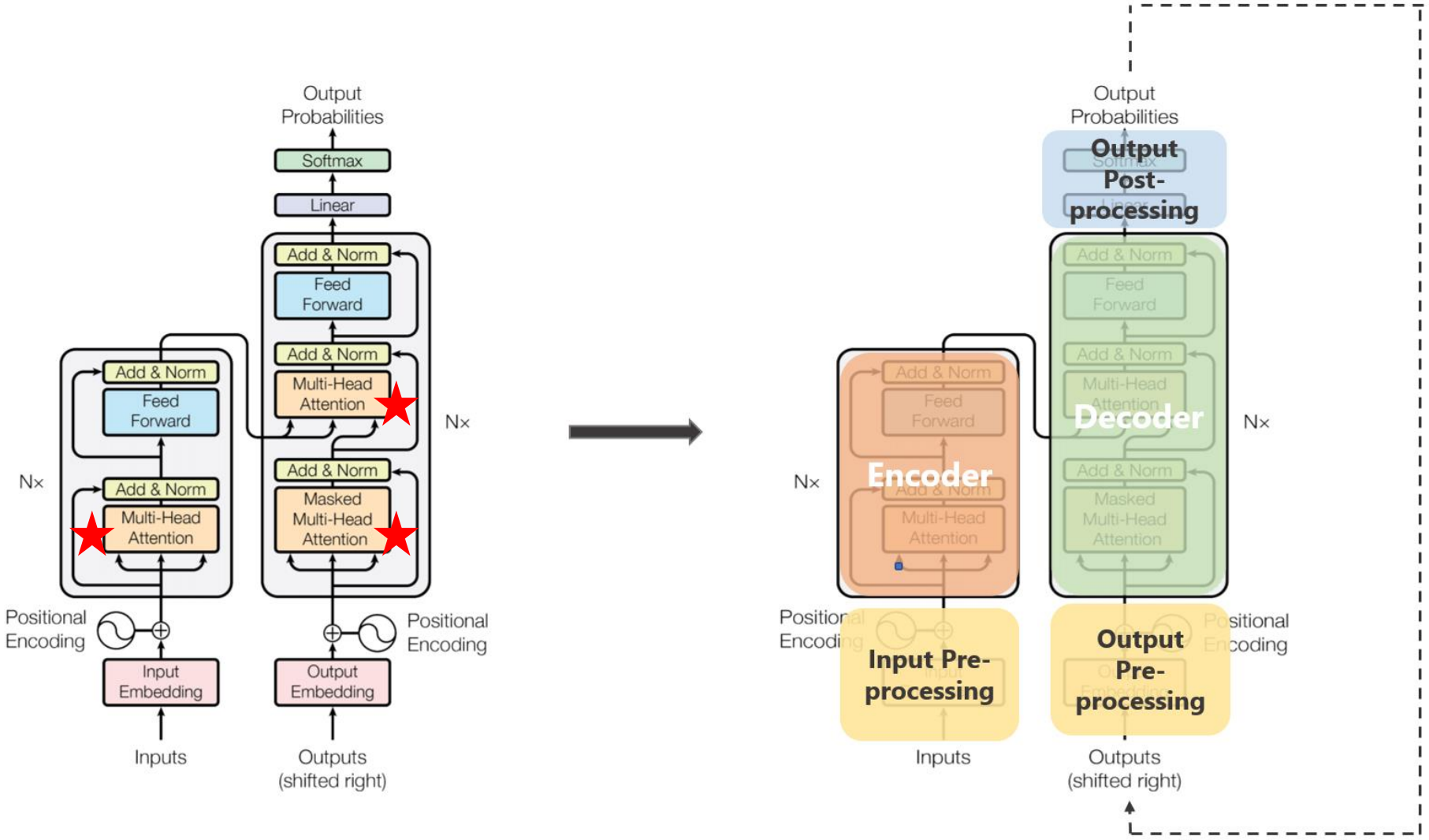


Each row of the Attention Matrix captures:

- The meaning of the word (token).
- The position of the word.
- The interaction with the other words.

In the Self-Attention Q = K = V are the embeddings of the word of the sentence.

Transformers

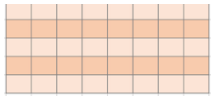


From https://towardsdatascience.com/transformers-89034557de14

Multi-Head Attention (MHA)

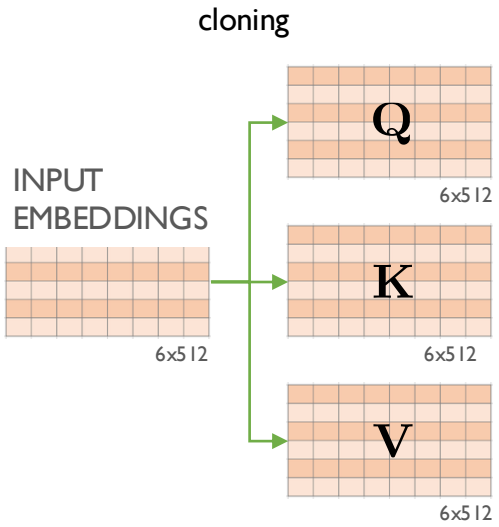
Multi-Head Attention (MHA)

INPUT
EMBEDDINGS

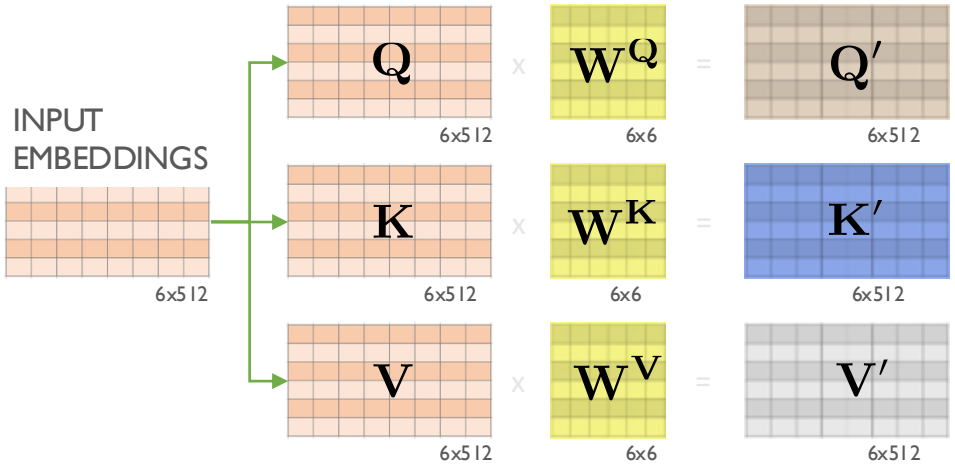


6x512

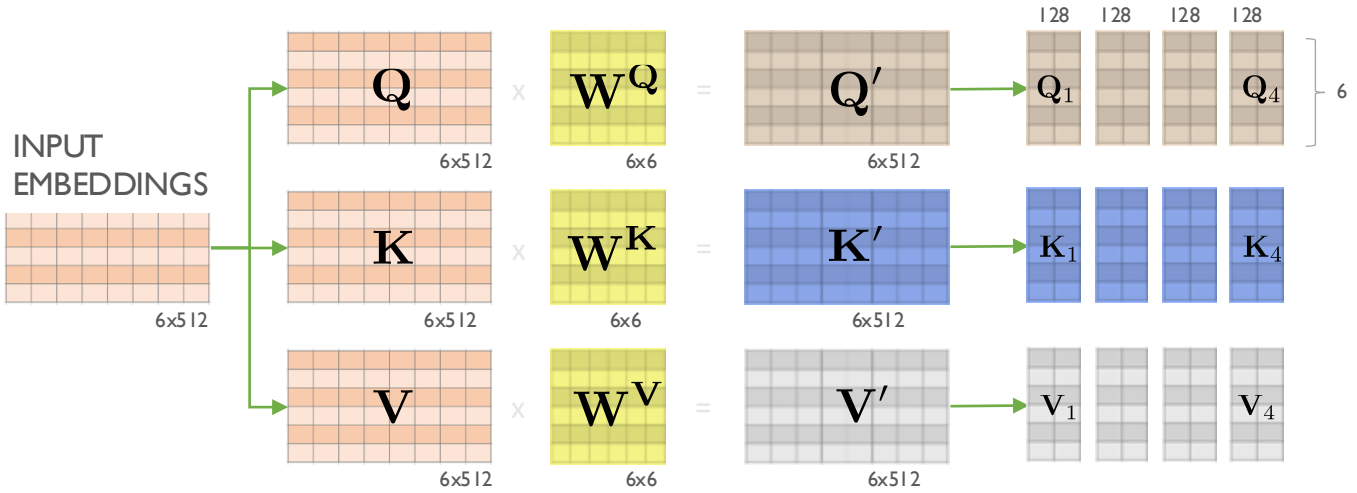
Multi-Head Attention (MHA)



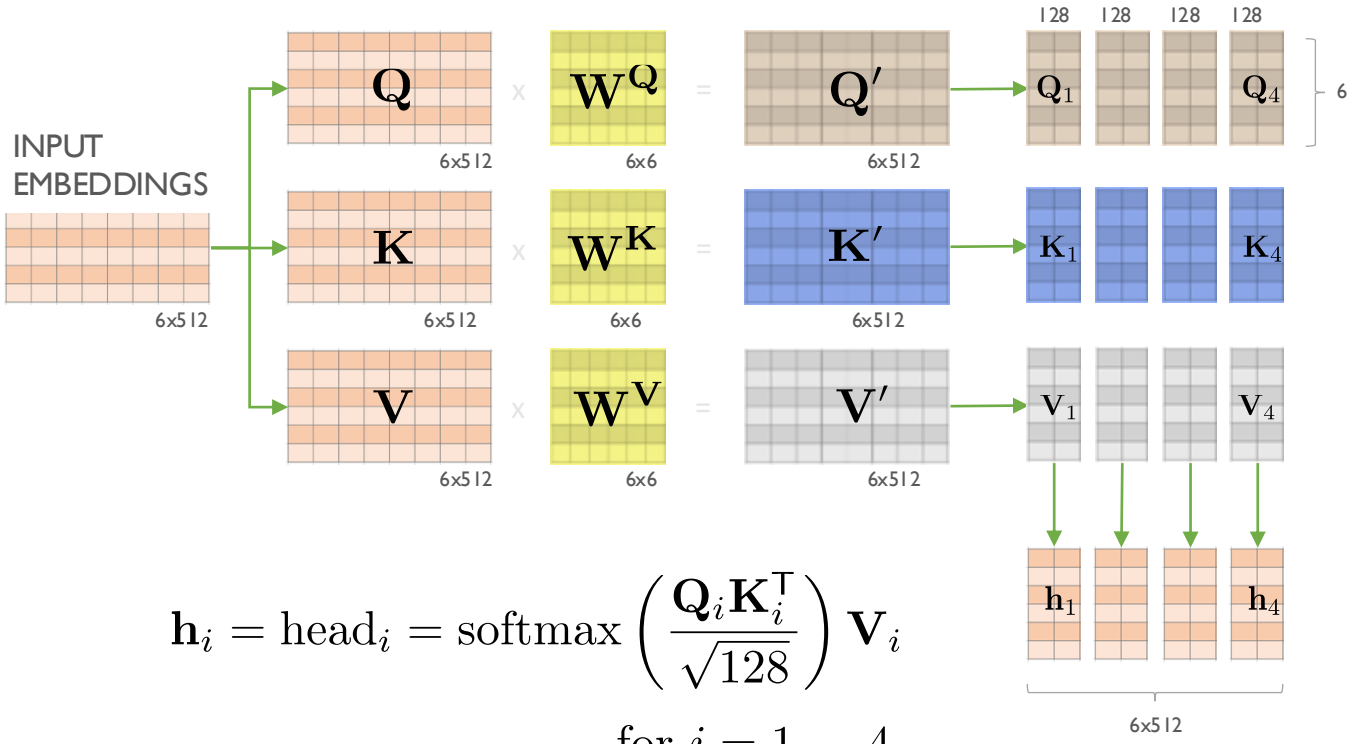
Multi-Head Attention (MHA)



Multi-Head Attention (MHA)



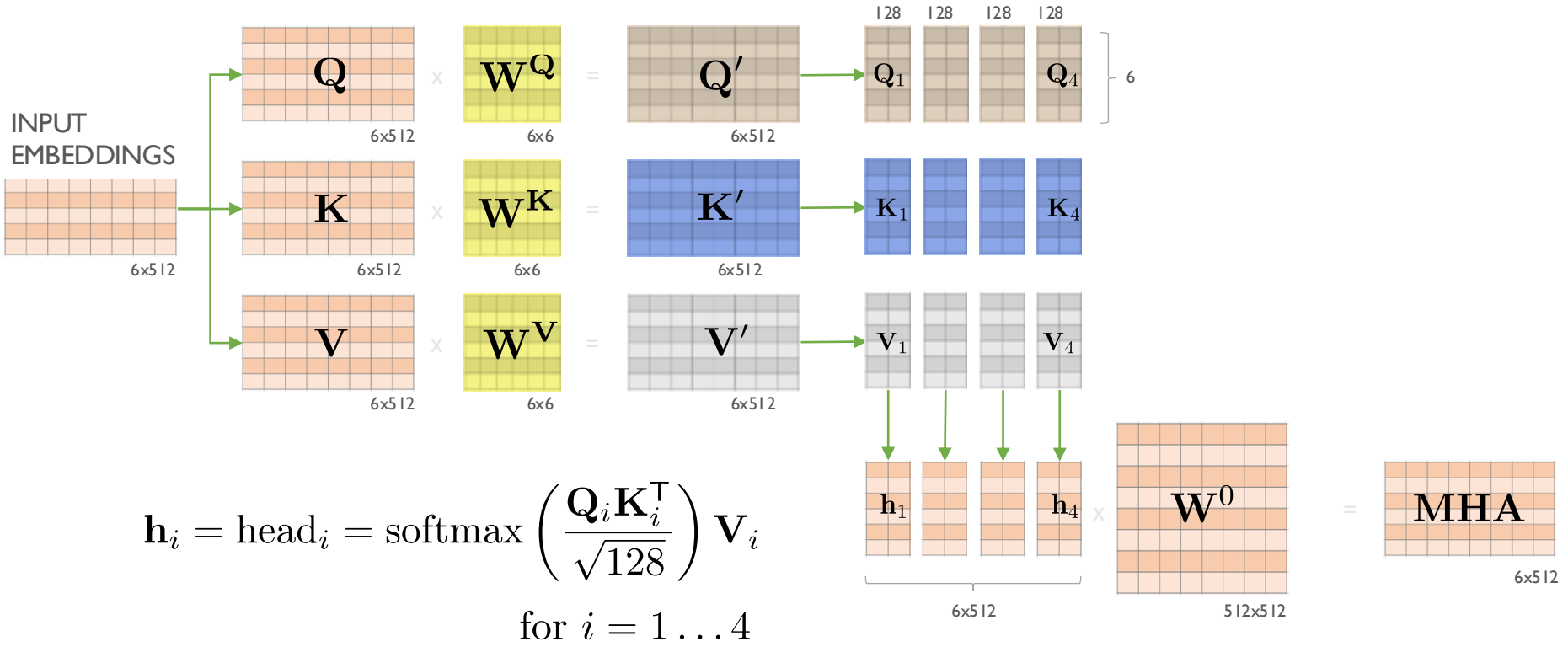
Multi-Head Attention (MHA)



$$\mathbf{h}_i = \text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{128}}\right)\mathbf{V}_i$$

$$\text{for } i = 1 \ldots 4$$

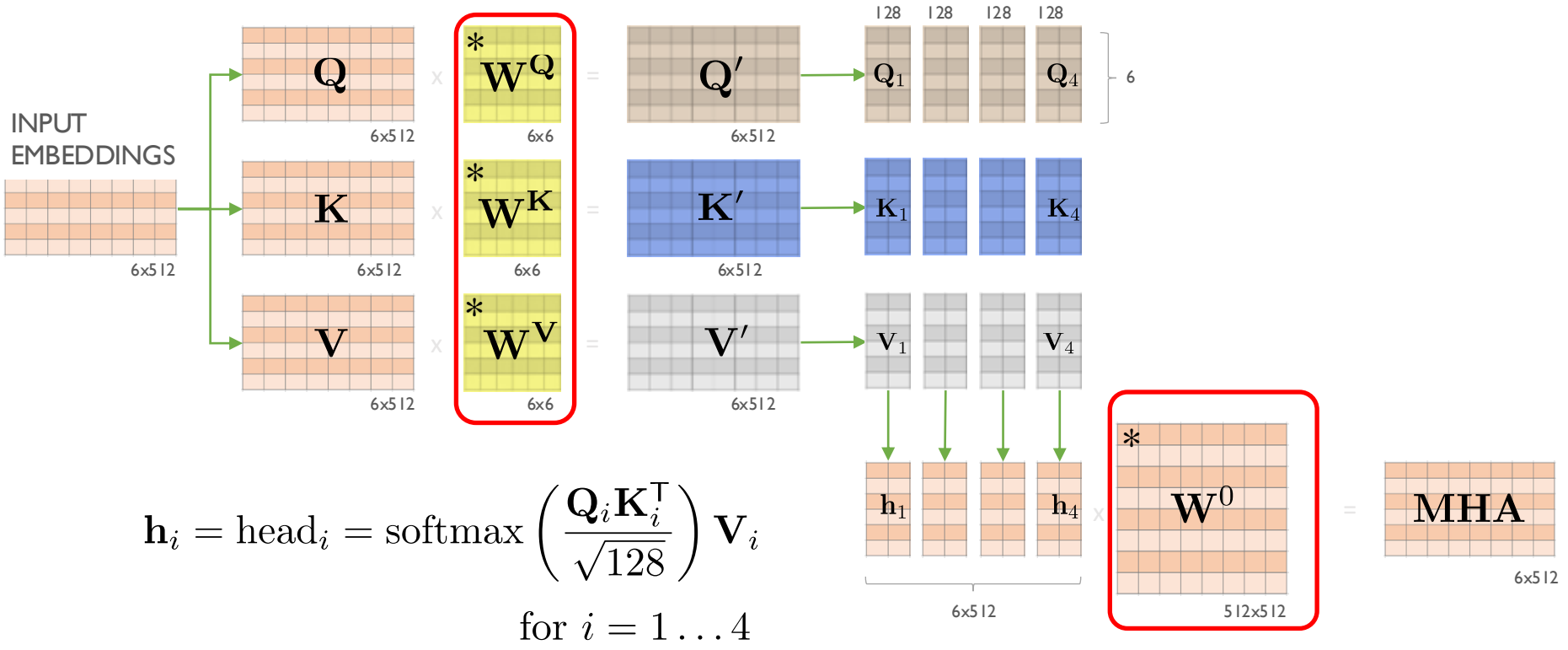
Multi-Head Attention (MHA)



$$\mathbf{h}_i = \text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{128}}\right)\mathbf{V}_i$$

$$\text{for } i = 1 \ldots 4$$

Multi-Head Attention (MHA)

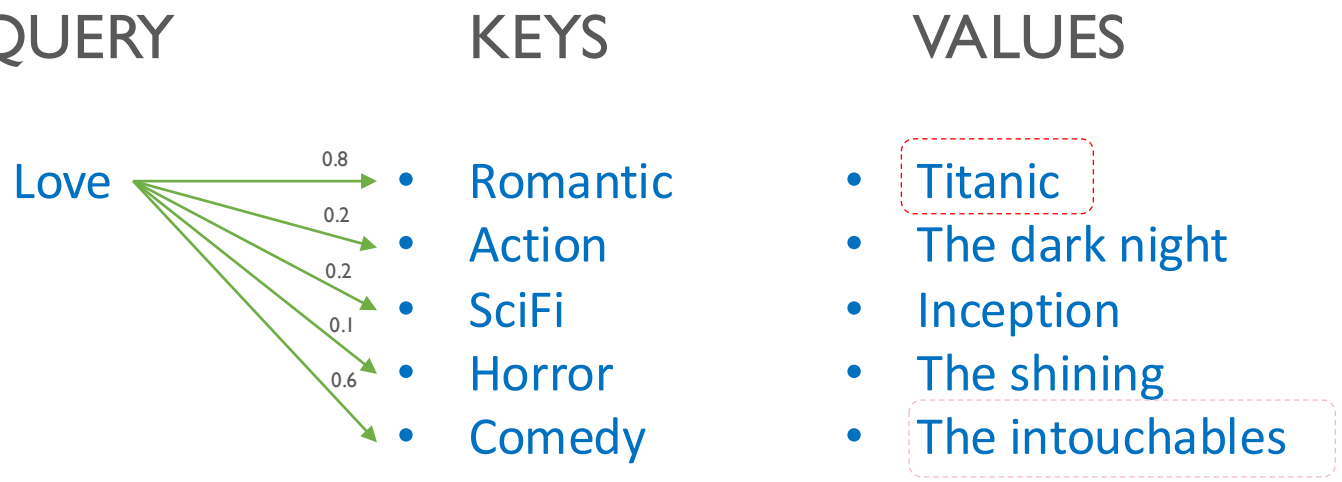


$$\mathbf{h}_i = \text{head}_i = \text{softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^\top}{\sqrt{128}}\right)\mathbf{V}_i$$

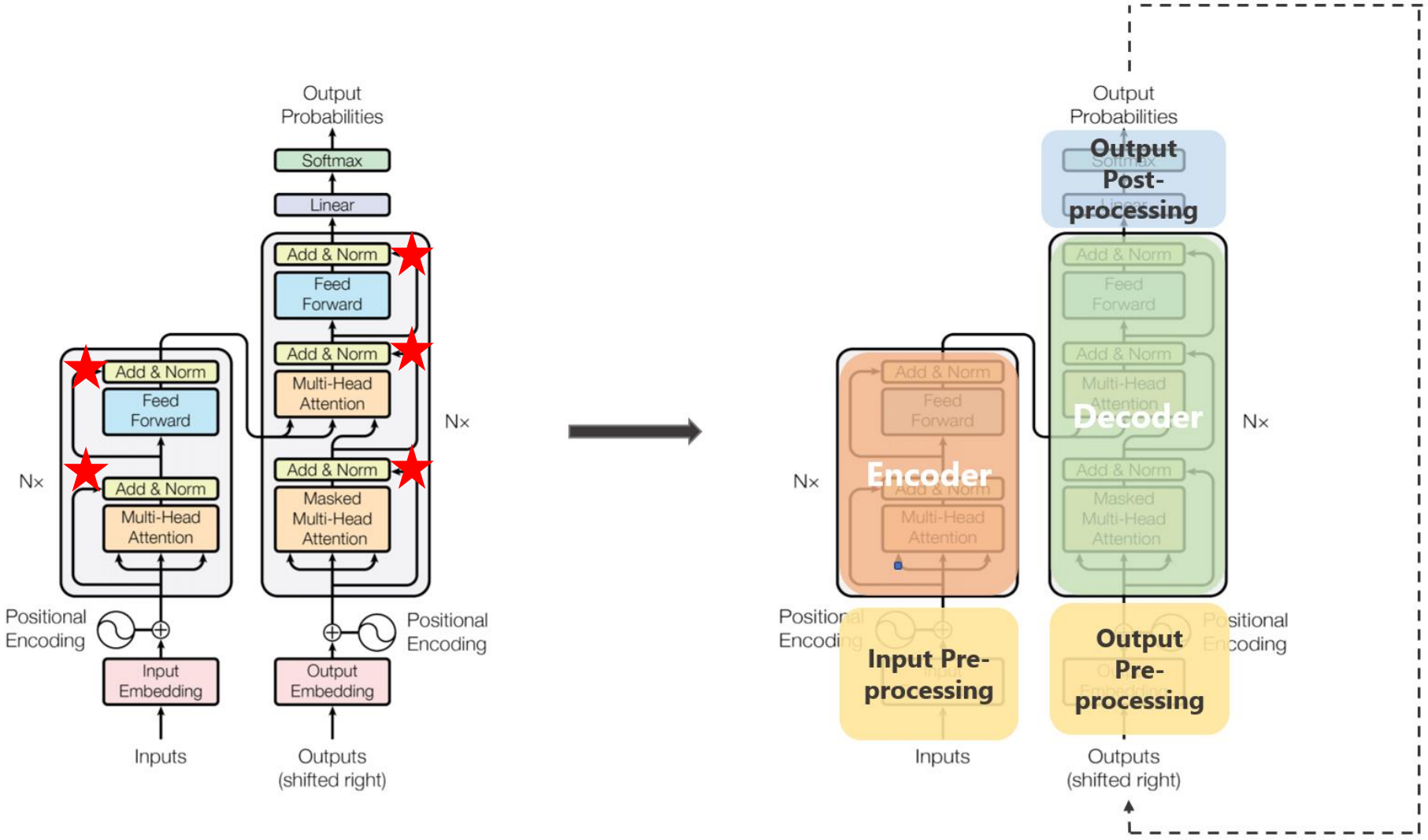
$$\text{for } i = 1 \ldots 4$$

* Learnable in a training process

Query, Key and Value

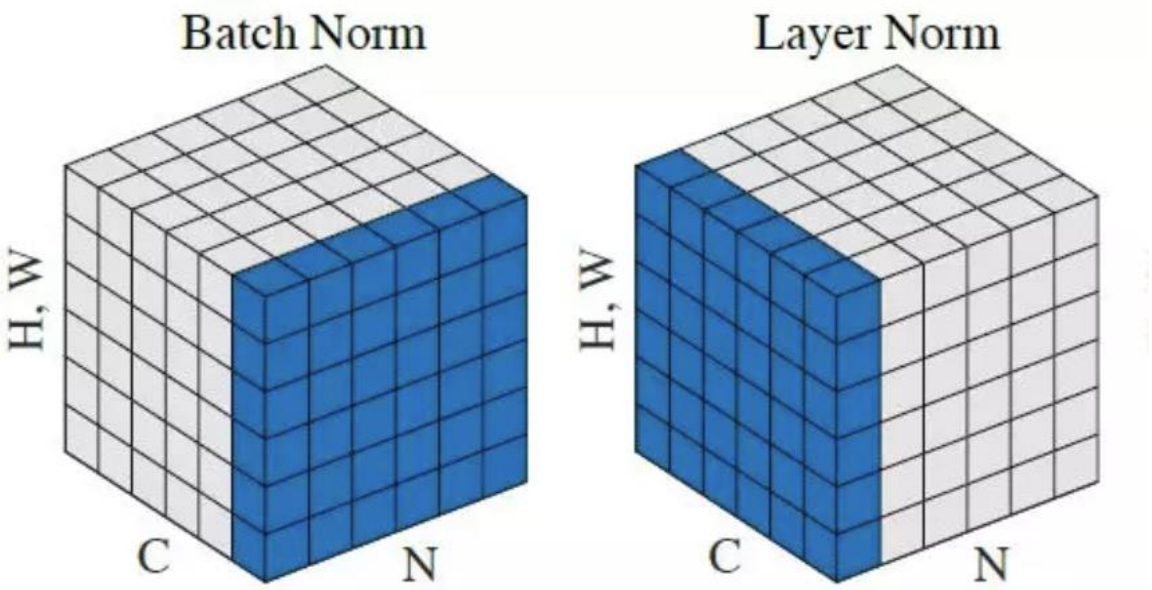


Transformers



From https://towardsdatascience.com/transformers-89034557de14

Layer Normalization

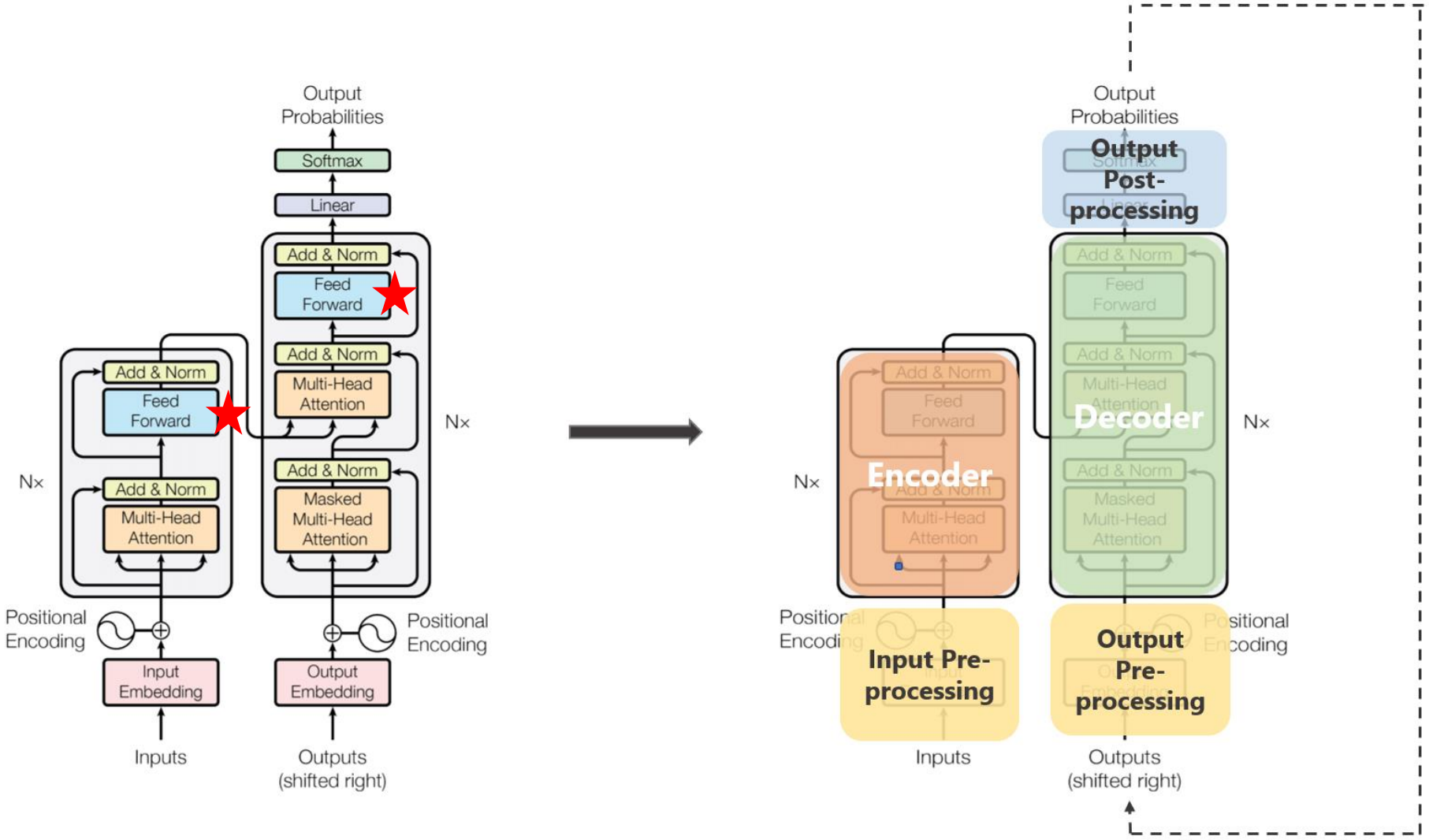


For images of HxW pixels and C channels

$$\hat{\mathbf{x}}_j = \gamma \frac{\mathbf{x}_j - \mu_j}{\sqrt{\sigma_j + \epsilon}} + \beta$$

From https://paperswithcode.com/method/layer-normalization

Transformers



From https://towardsdatascience.com/transformers-89034557de14

Feed Forward – Neural Network

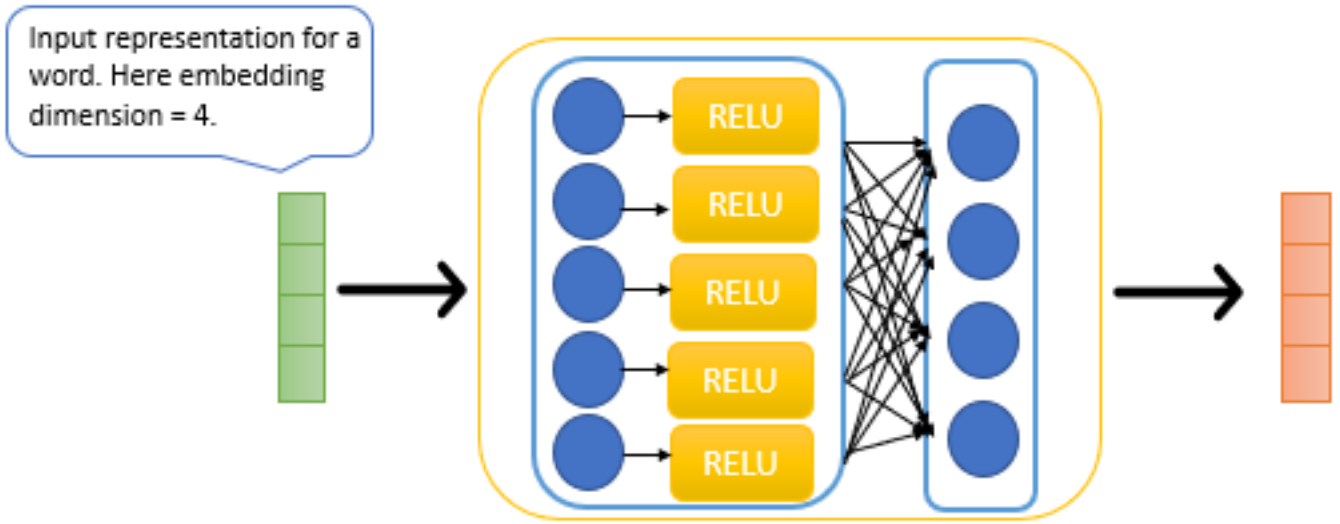
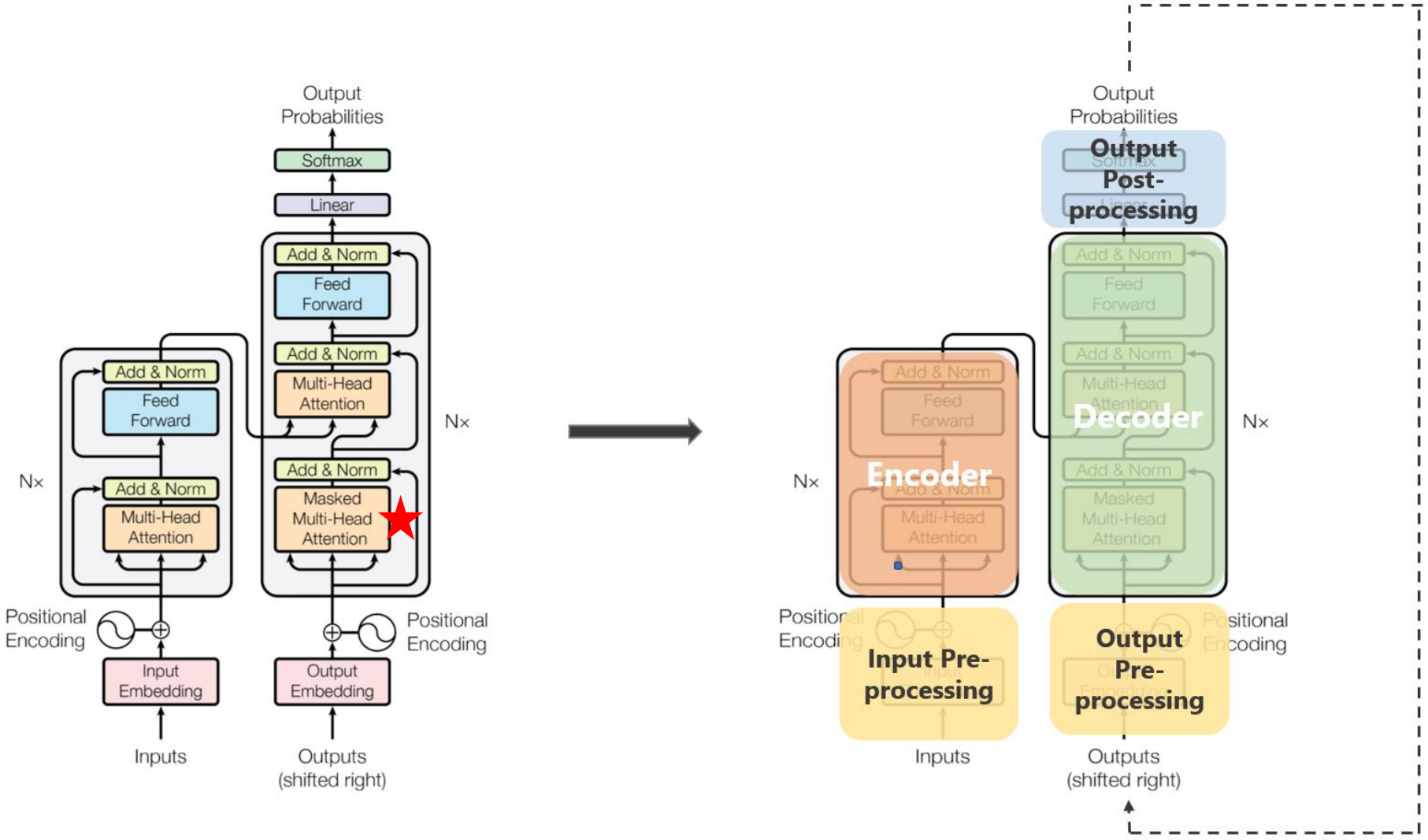


Fig 16: Feed forward neural network. Here input to inner layer is of size = 4, inner layer has 5 neurons and output layer has 4 neurons. Actual transformer model has 512 neurons in output layer and 2048 in inner layer

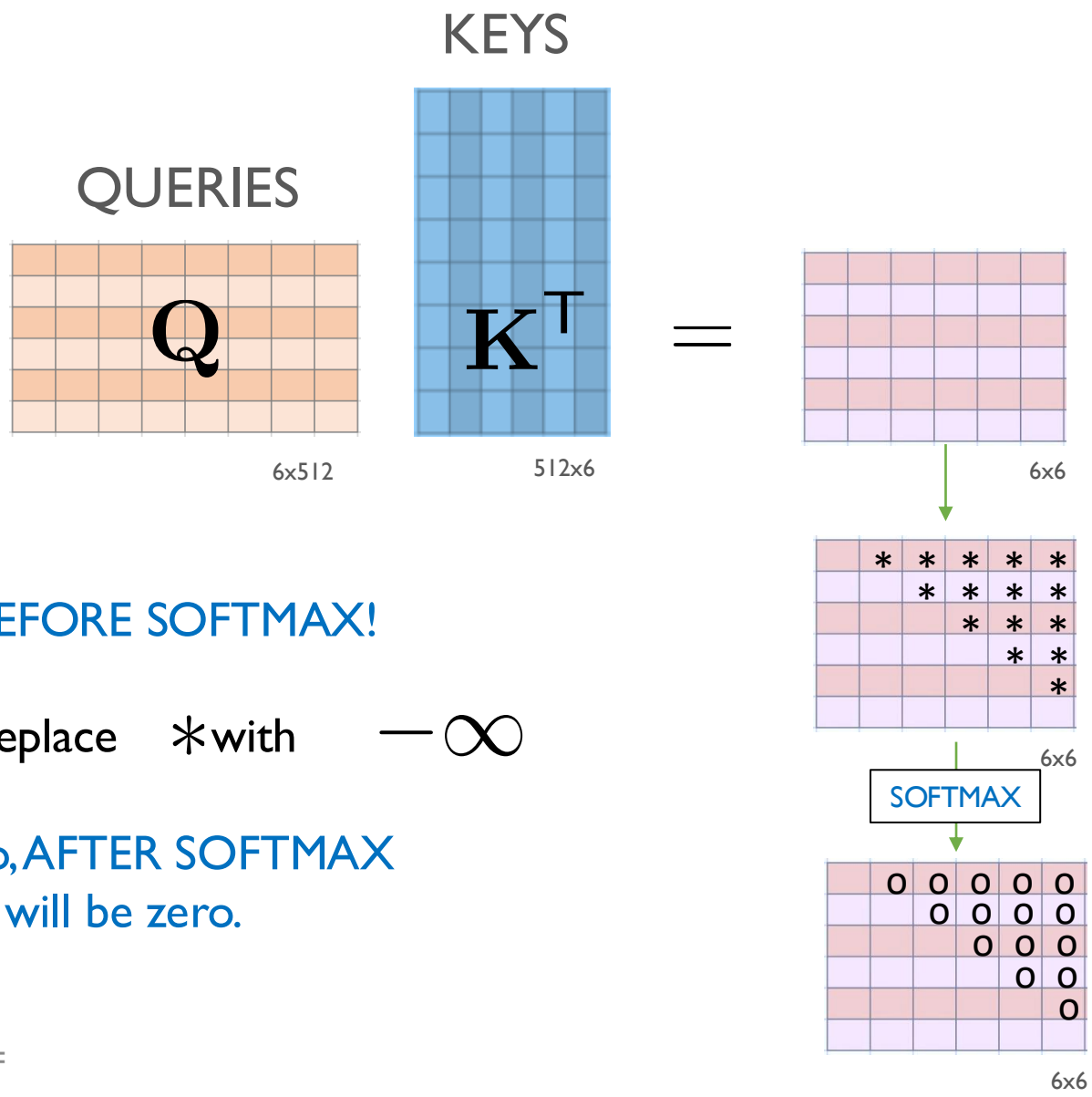
https://raviteja-ganta.github.io/attention-is-all-you-need-transformers

Transformers



From https://towardsdatascience.com/transformers-89034557de14

Masked Multi-Head Attention



BEFORE SOFTMAX!

Replace $*$ with $-\infty$

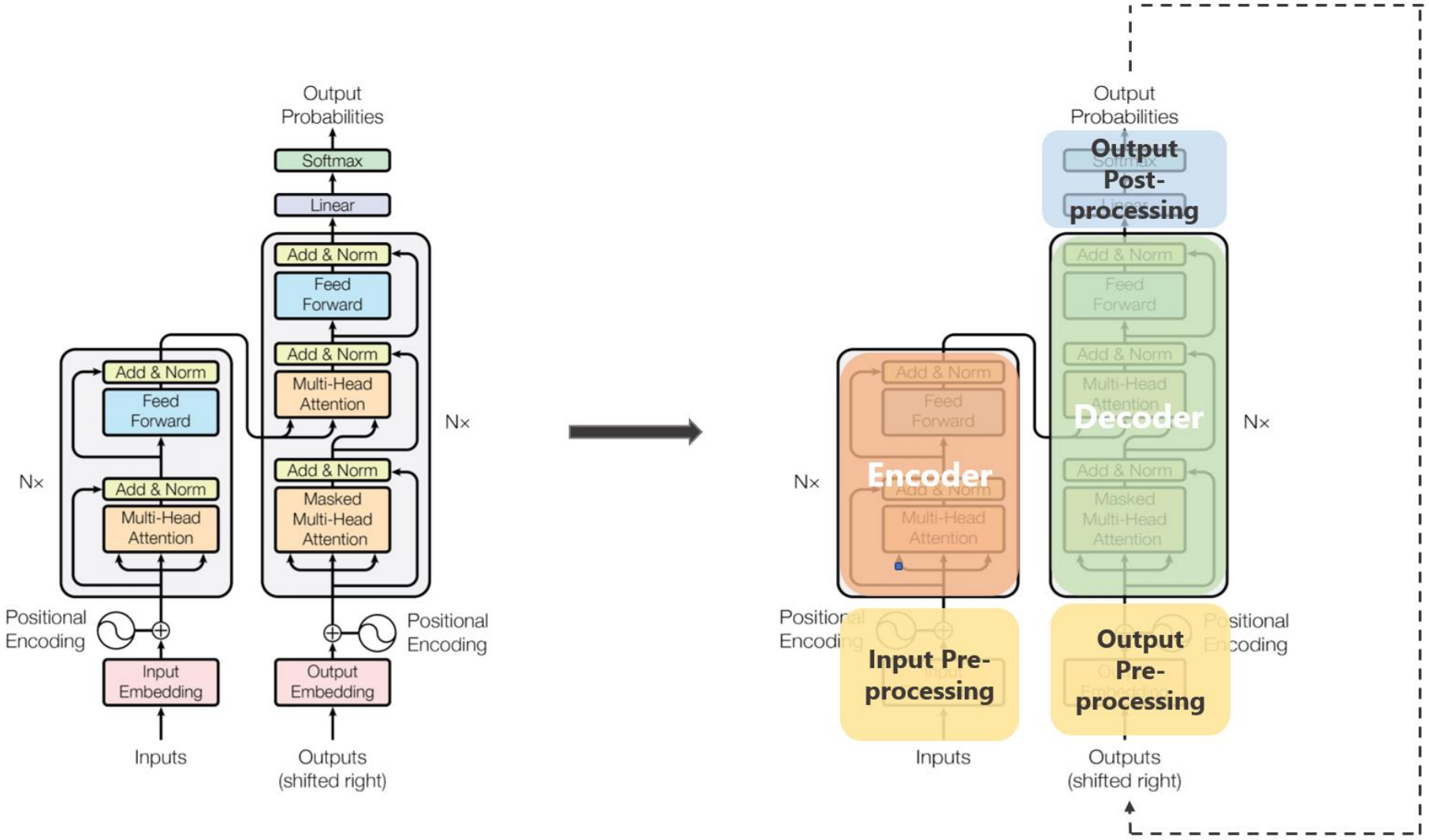
so, AFTER SOFTMAX
it will be zero.

CAUSALITY: The model must not be able to see future words.

When word i arrives, we will only have attention values with i, i-1, … 1

It is applied in the softmax's of the heads

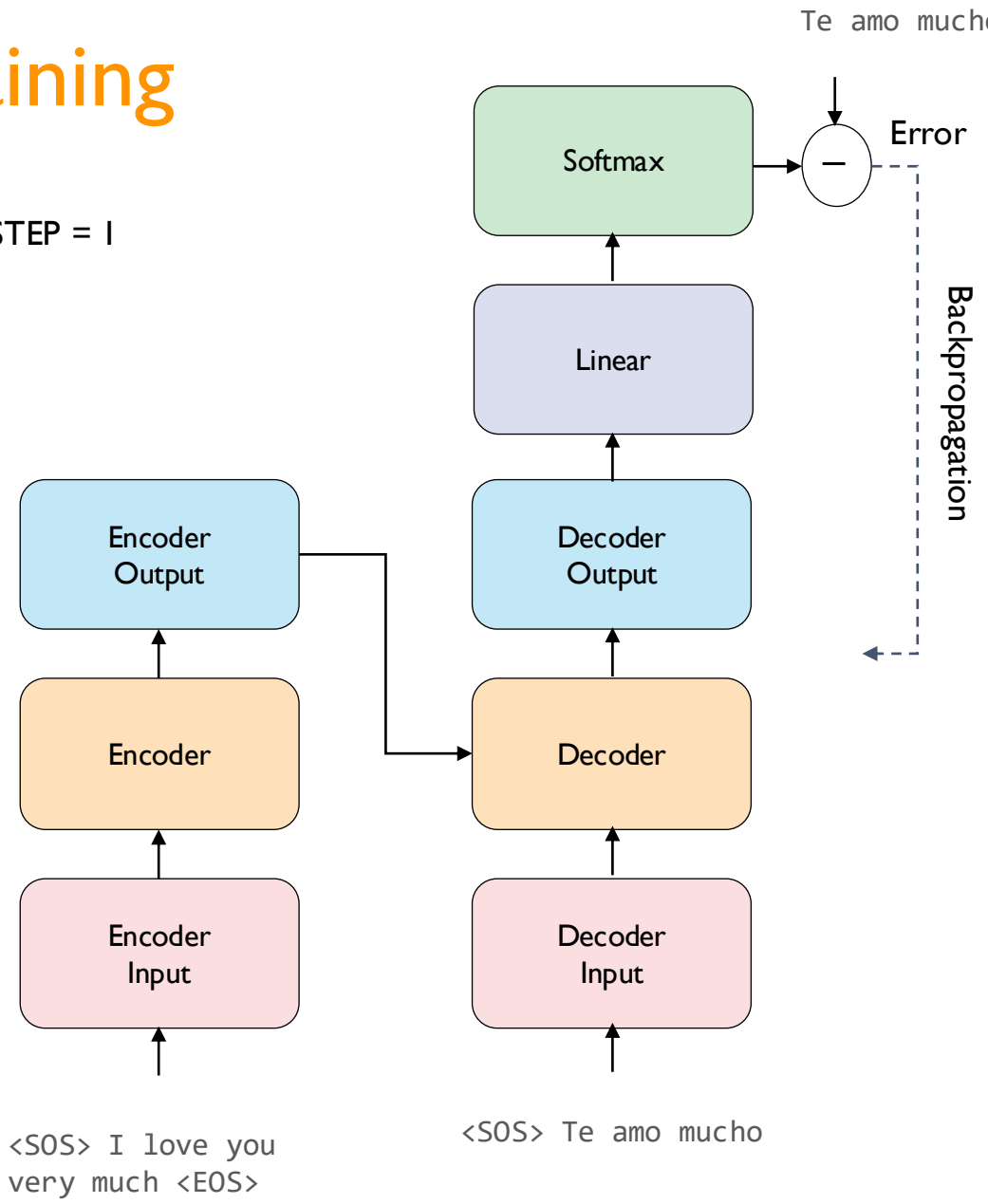
The Whole Model (for translation)

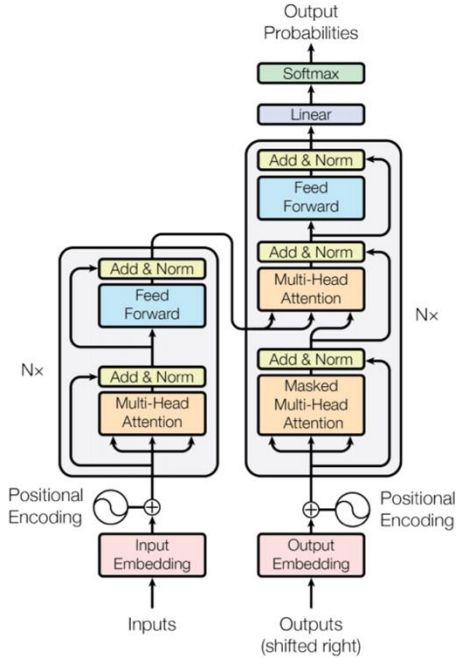


From https://towardsdatascience.com/transformers-89034557de14

Training

TIME STEP = 1

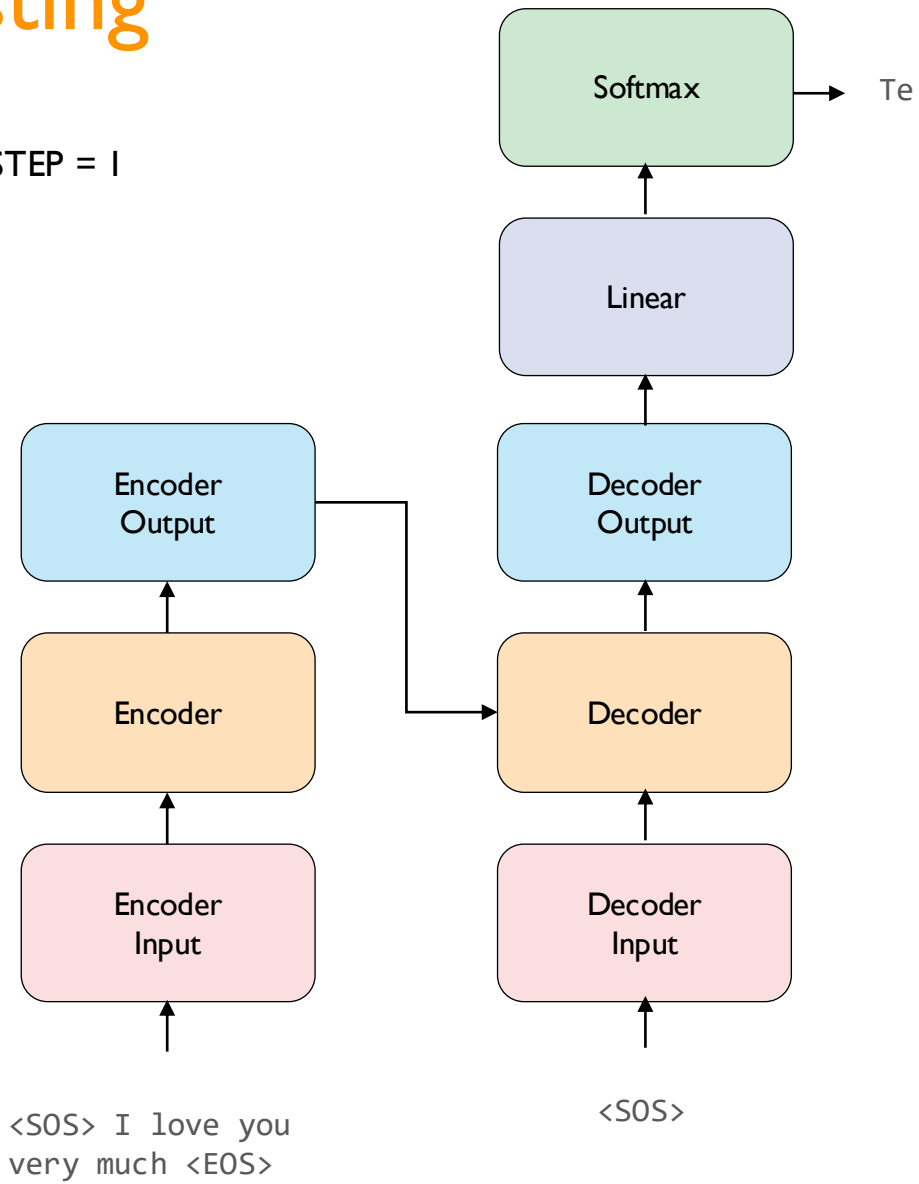


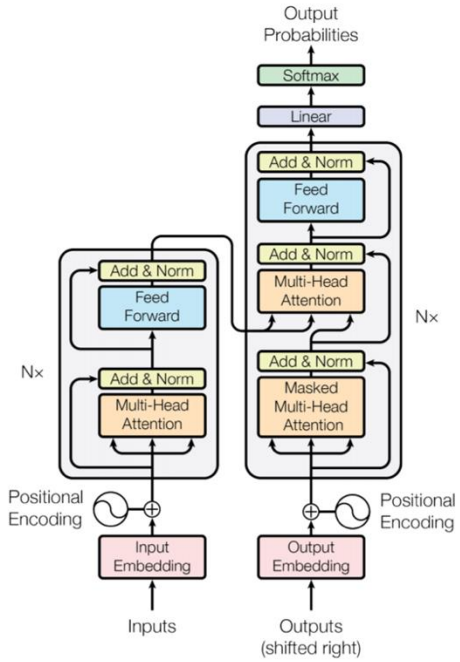


Adapted from: Umar Jamil: Attention is all you need (Transformer) – Model explanation.
https://github.com/hkproj/transformer-from-scratch-notes

Testing

TIME STEP = 1

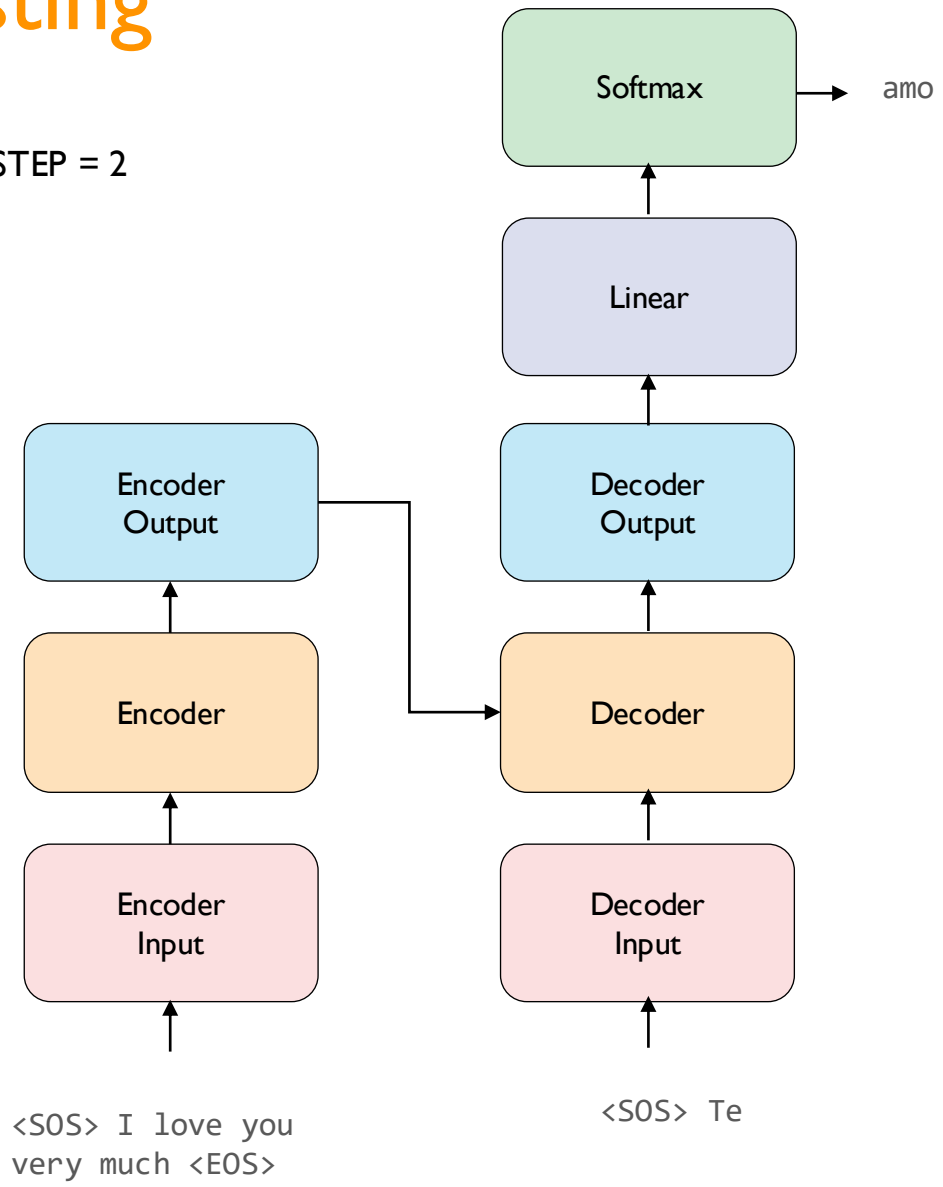


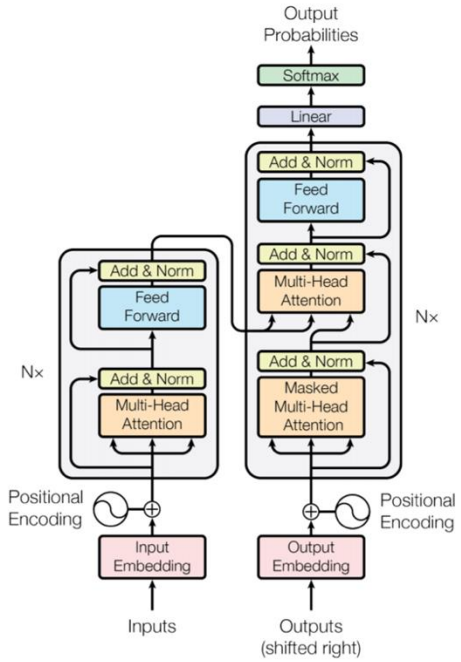


Adapted from: Umar Jamil: Attention is all you need (Transformer) – Model explanation.
https://github.com/hkproj/transformer-from-scratch-notes

Testing

TIME STEP = 2

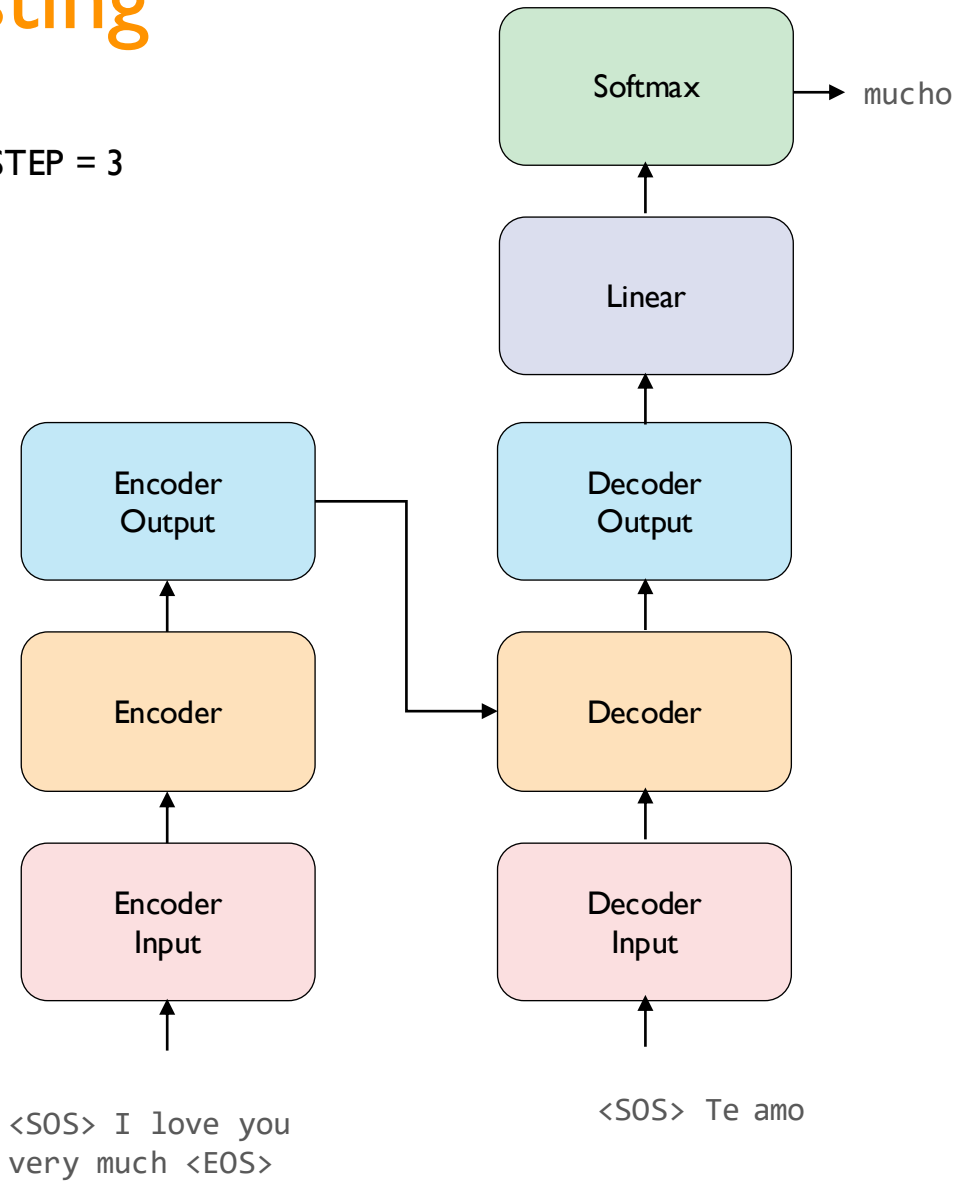


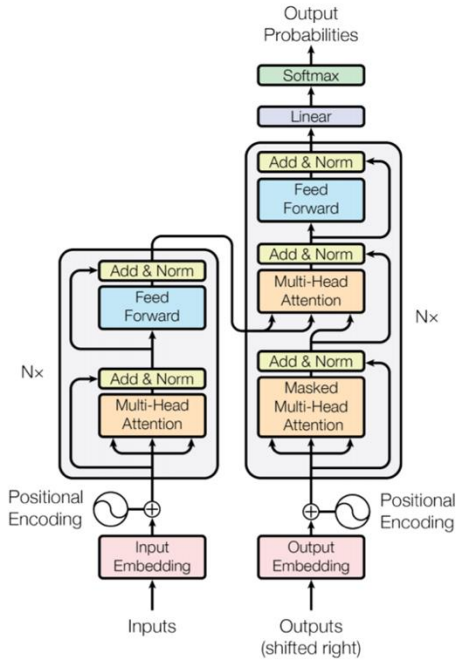


Adapted from: Umar Jamil: Attention is all you need (Transformer) – Model explanation.
https://github.com/hkproj/transformer-from-scratch-notes

Testing

TIME STEP = 3

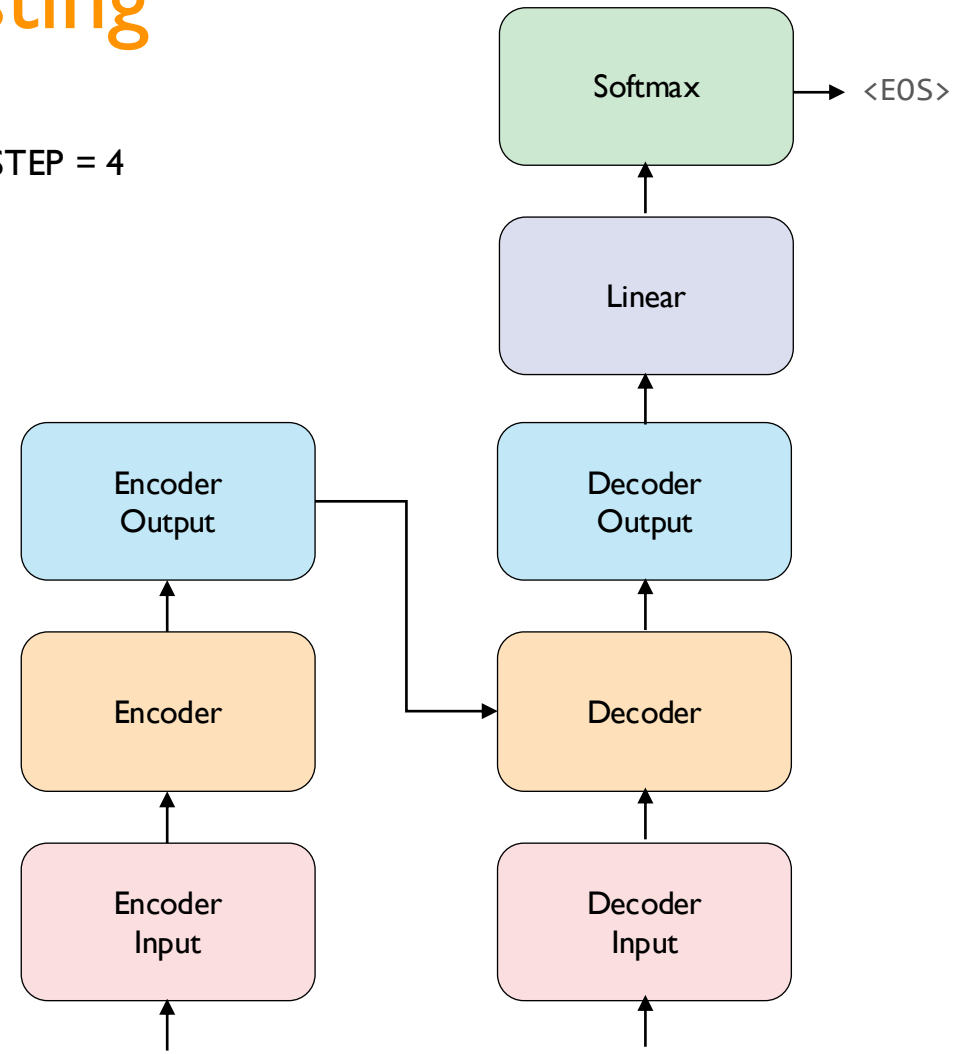


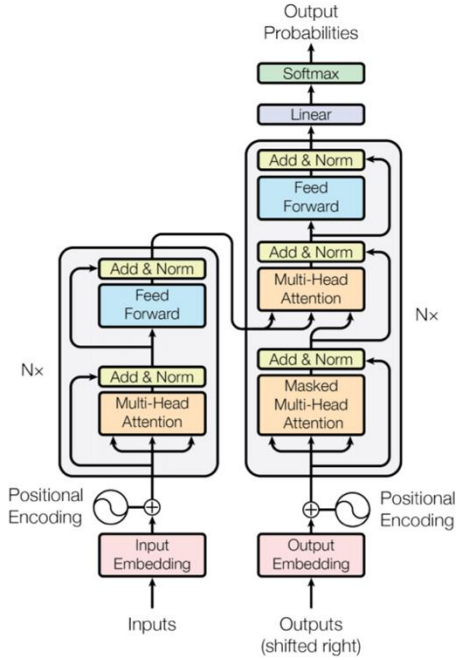


Adapted from: Umar Jamil: Attention is all you need (Transformer) – Model explanation.
https://github.com/hkproj/transformer-from-scratch-notes

Testing

TIME STEP = 4

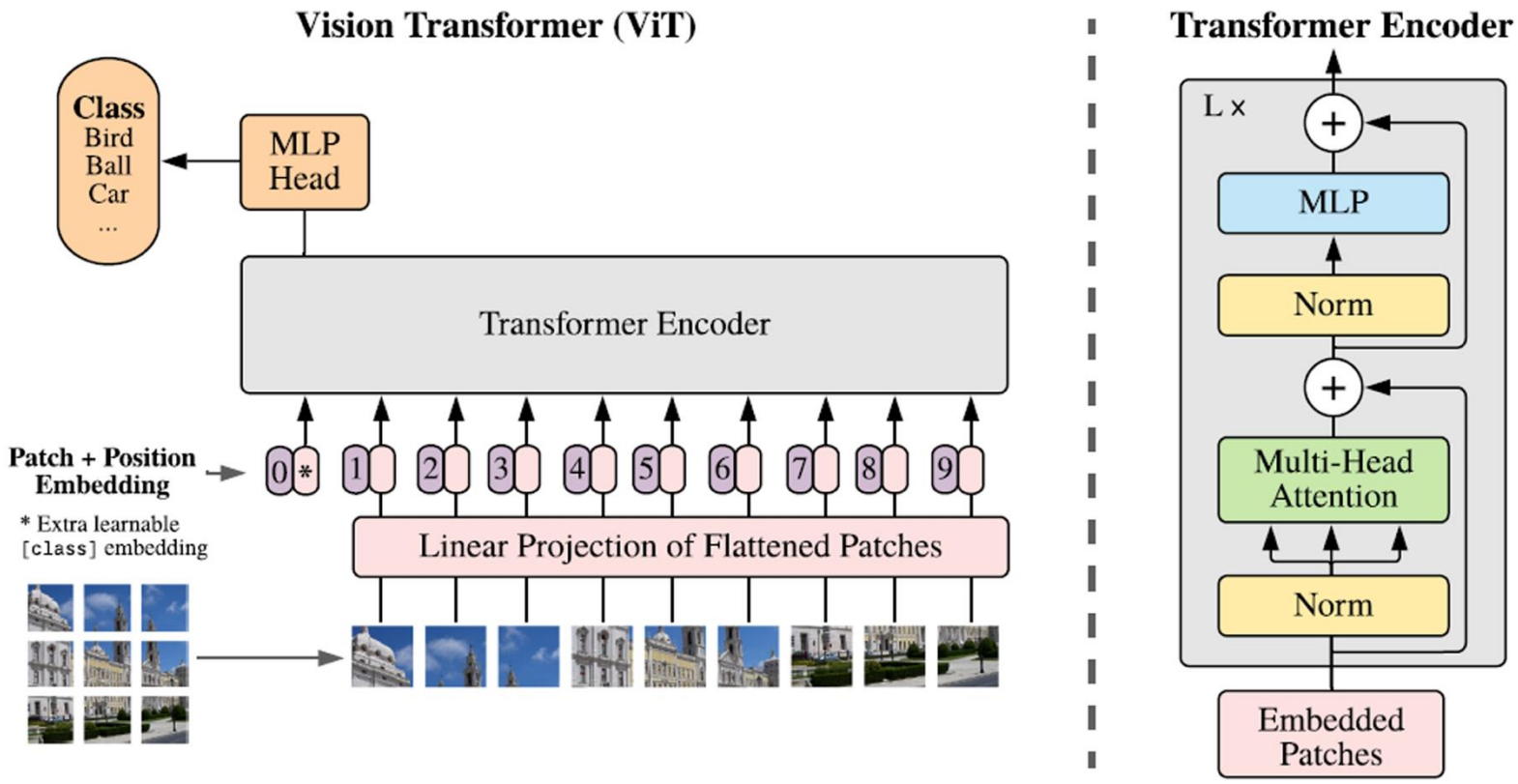




Adapted from: Umar Jamil: Attention is all you need (Transformer) – Model explanation.
https://github.com/hkproj/transformer-from-scratch-notes

Visual Transformer (ViT)

Visual Transformer (ViT)



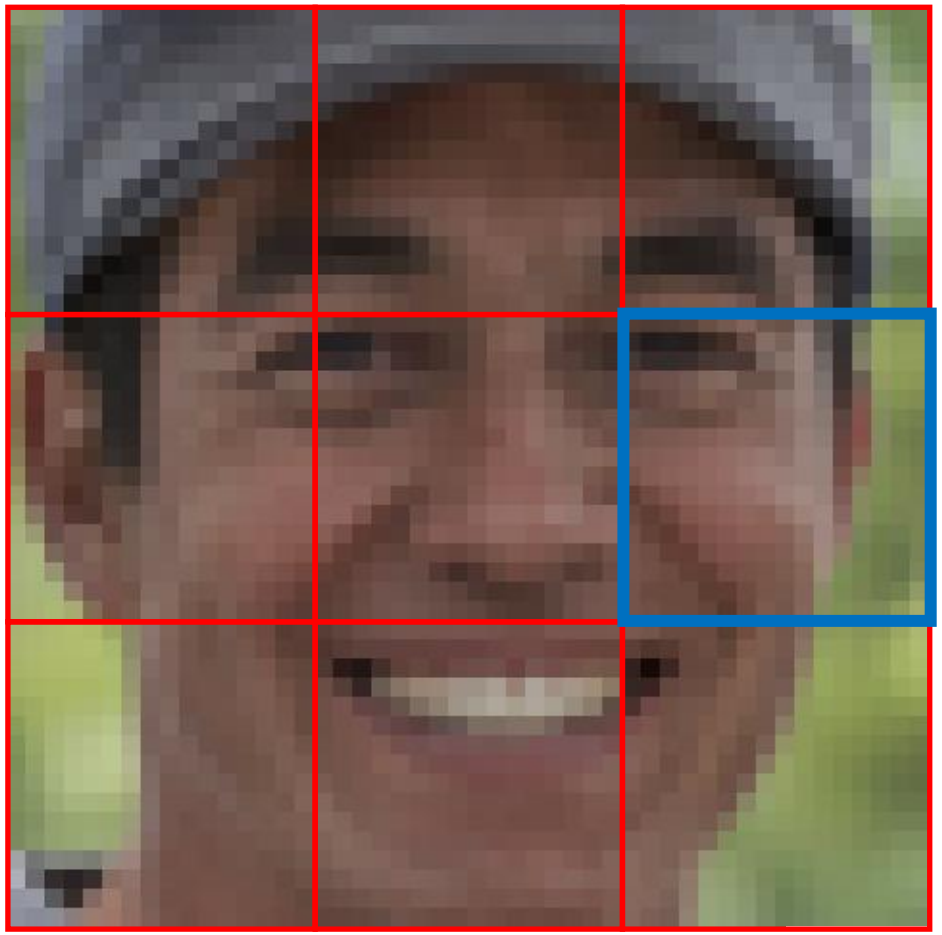
Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani et al. "An image is worth 16x16 words: Transformers for image recognition at scale." arXiv preprint arXiv:2010.11929 (2020).

Visual Transformer (ViT)



Example: ViT in Face Recognition

Visual Transformer (ViT)



48x48

Visual Transformer (ViT)



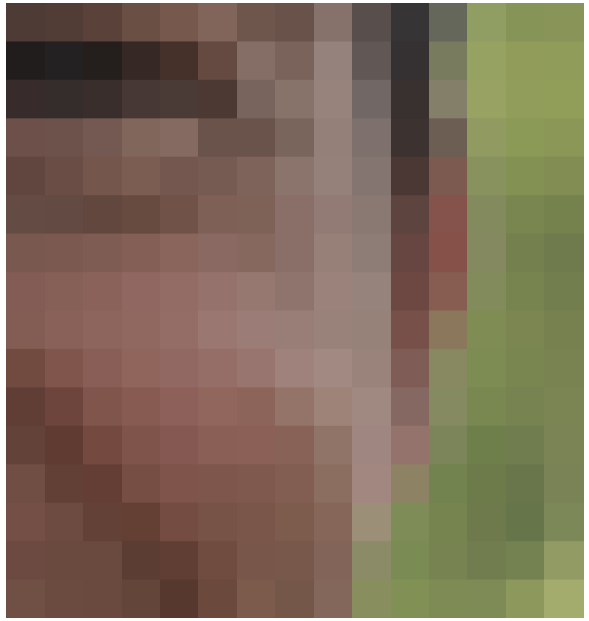
16x16

Visual Transformer (ViT)

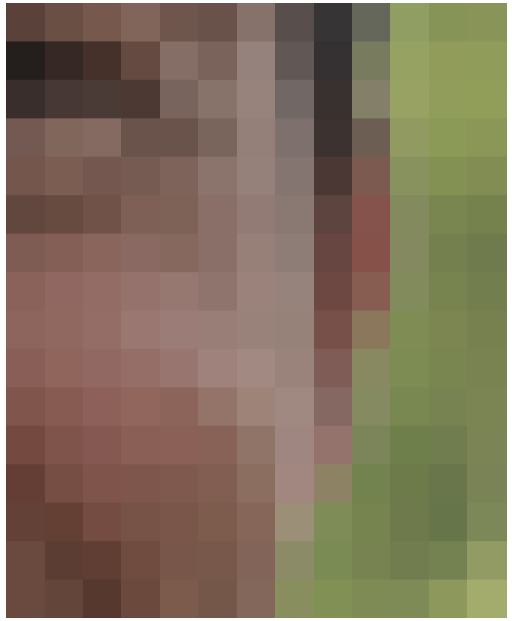


16x16

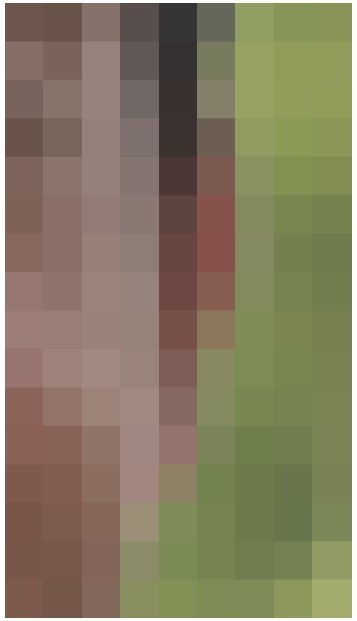
Visual Transformer (ViT)



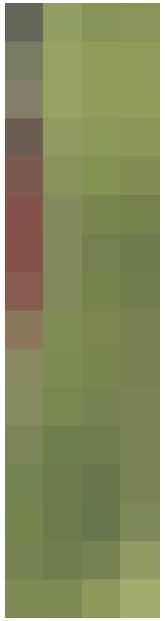
Visual Transformer (ViT)



Visual Transformer (ViT)



Visual Transformer (ViT)



Visual Transformer (ViT)

Visual Transformer (ViT)

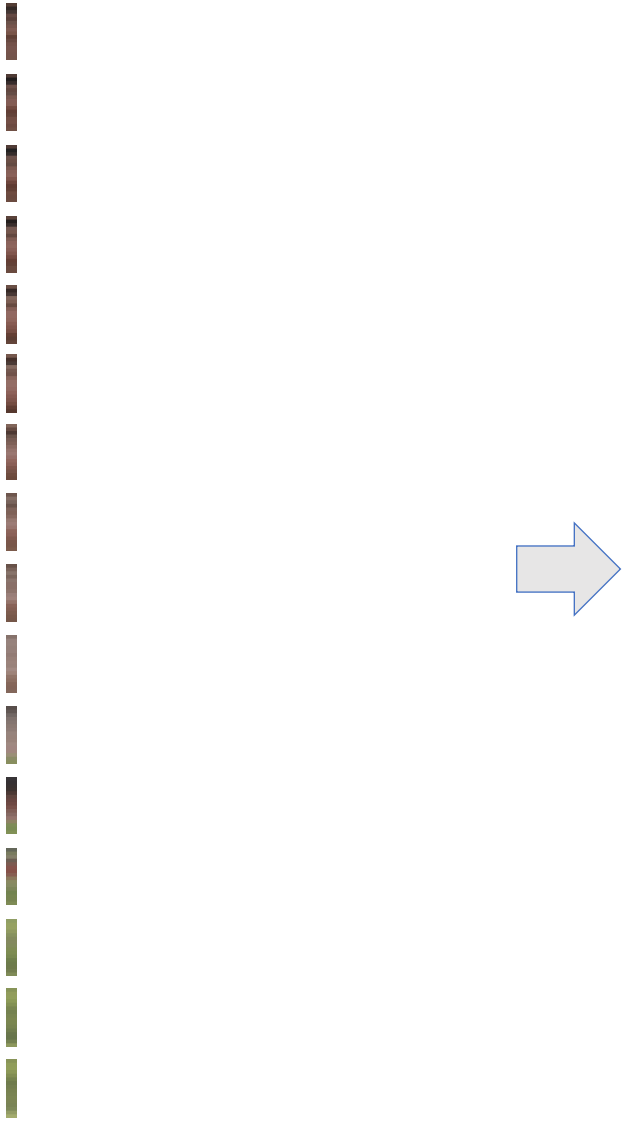
Visual Transformer (ViT)

256x1x3

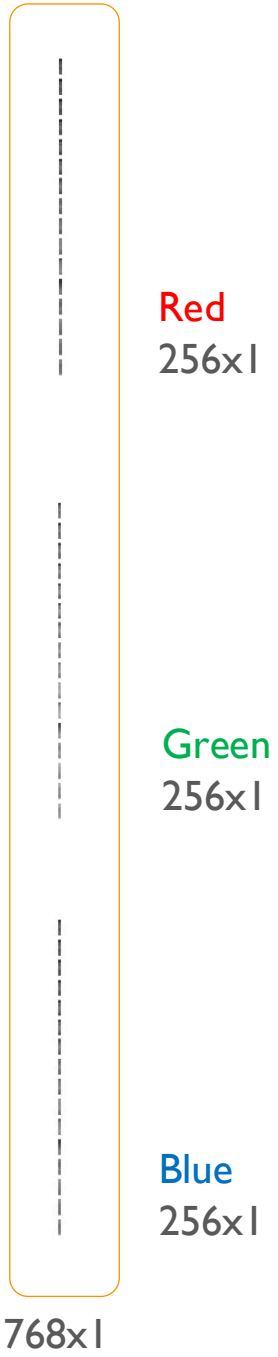
Visual Transformer (ViT)

256x1x3

Visual Transformer (ViT)



256x1x3



Red
256x1

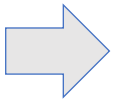
Green
256x1

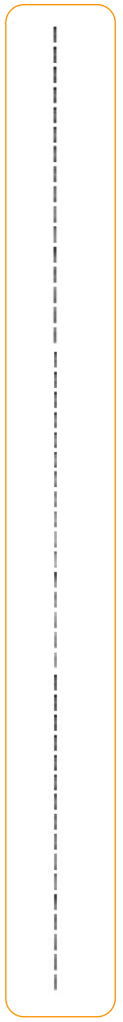
Blue
256x1

768x1

Visual Transformer (ViT)

256x1x3

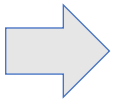


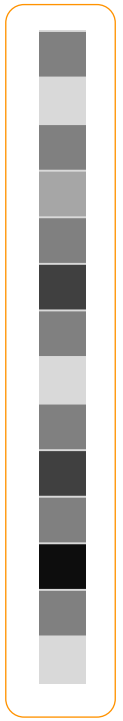


768x1

Visual Transformer (ViT)

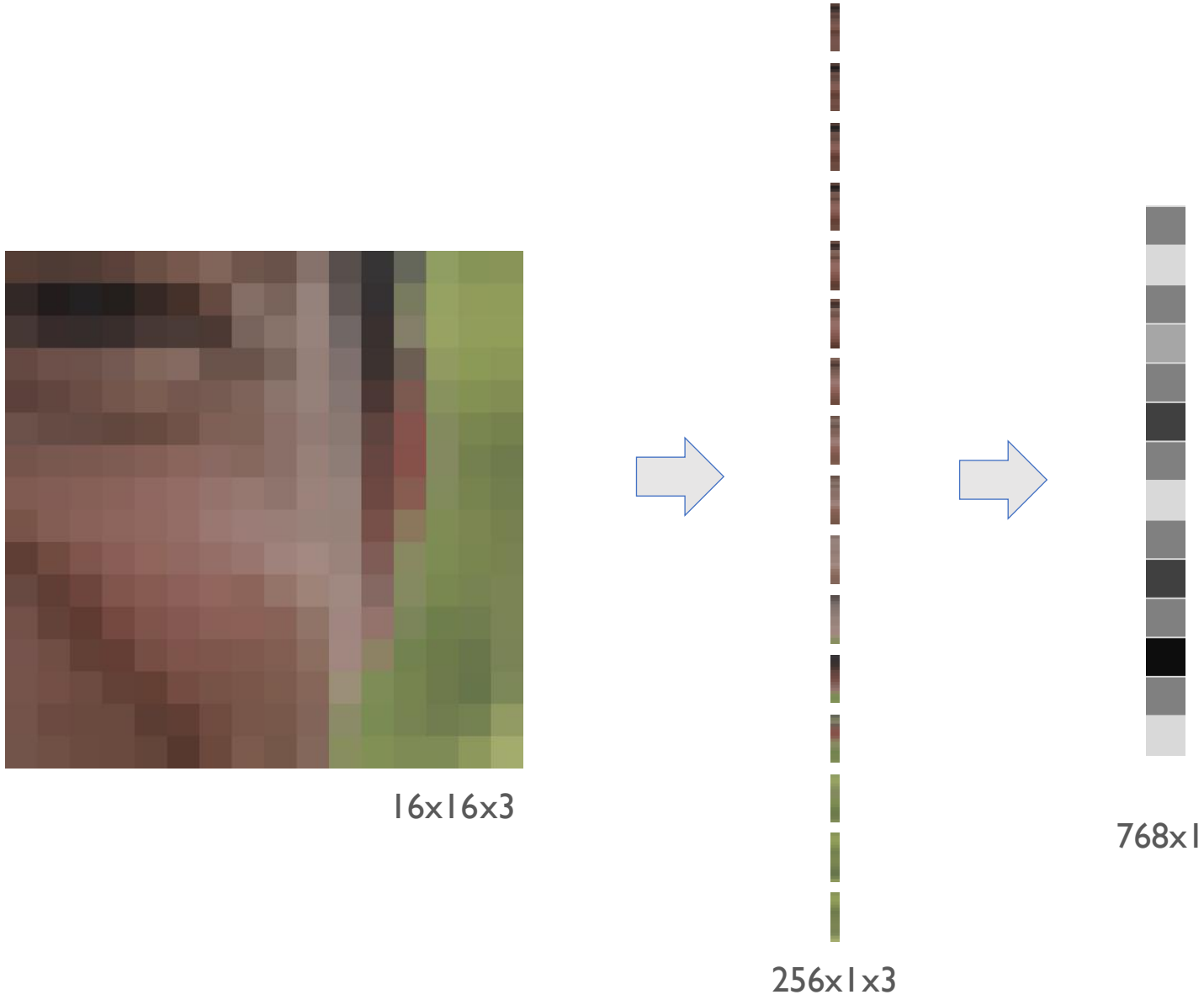
256x1x3





768x1

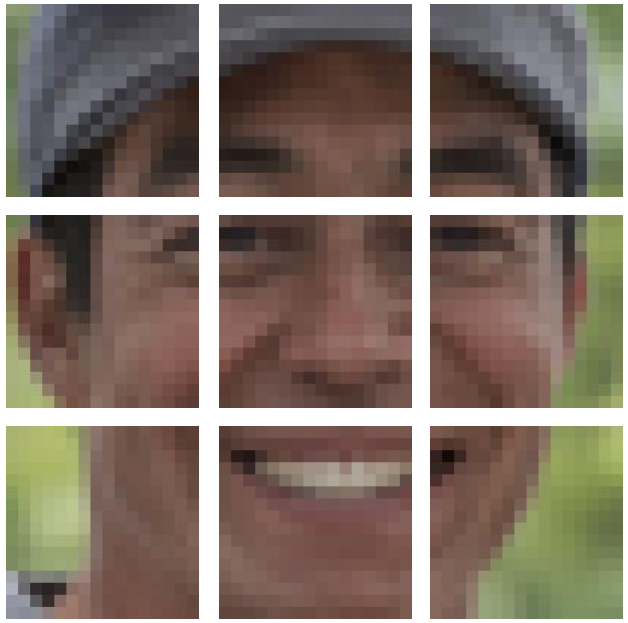
Visual Transformer (ViT)



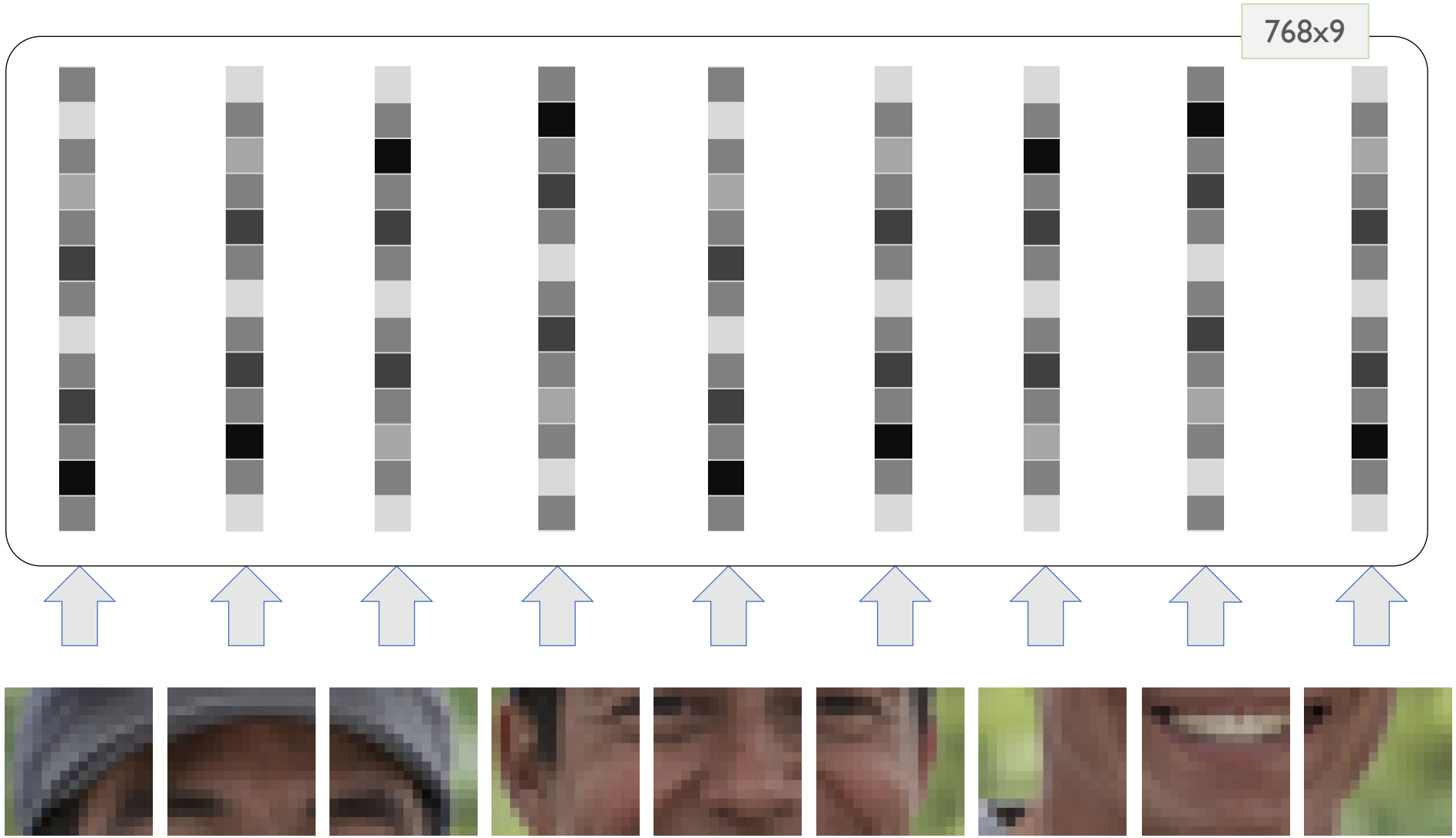
Visual Transformer (ViT)

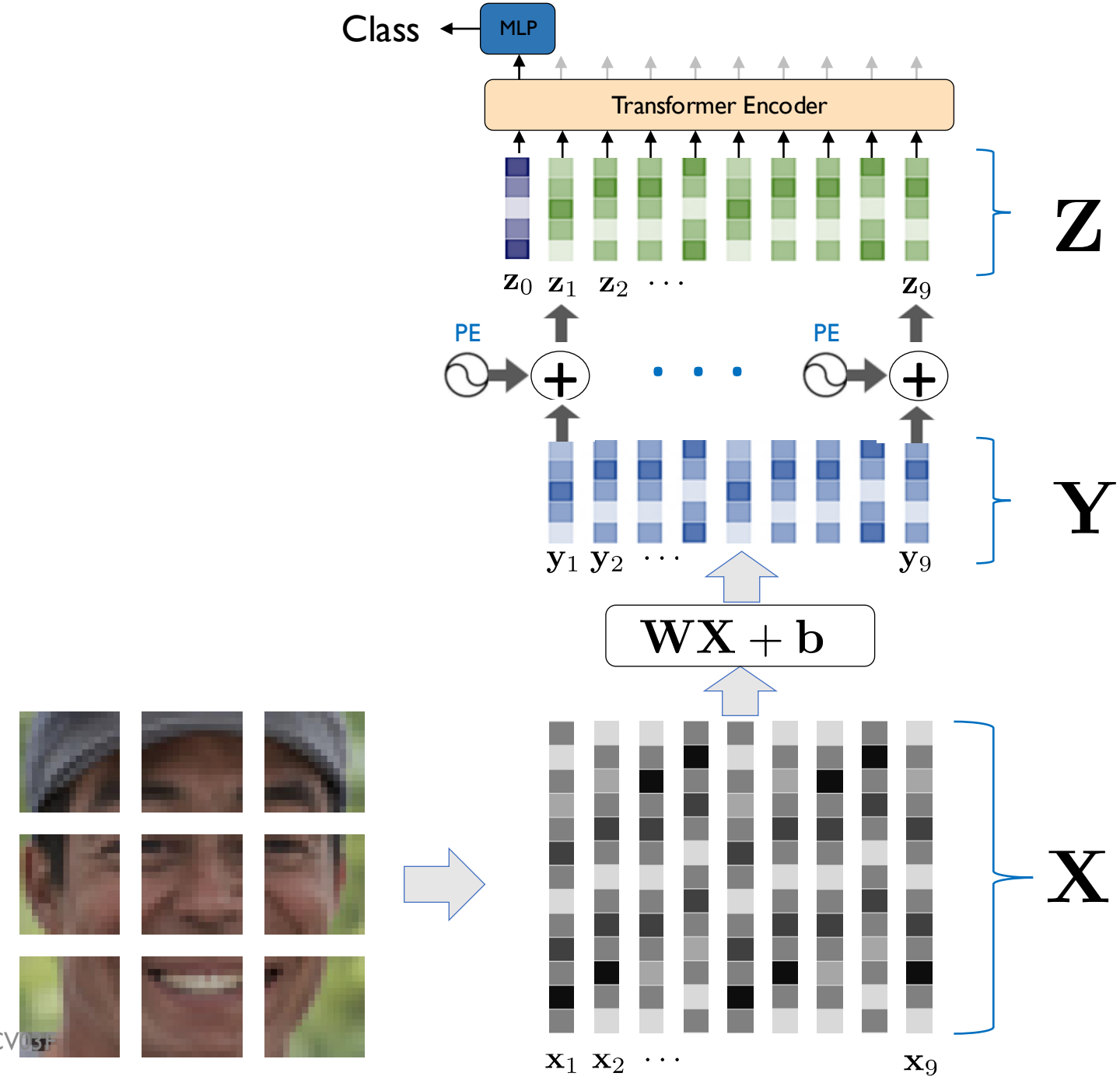


Visual Transformer (ViT)

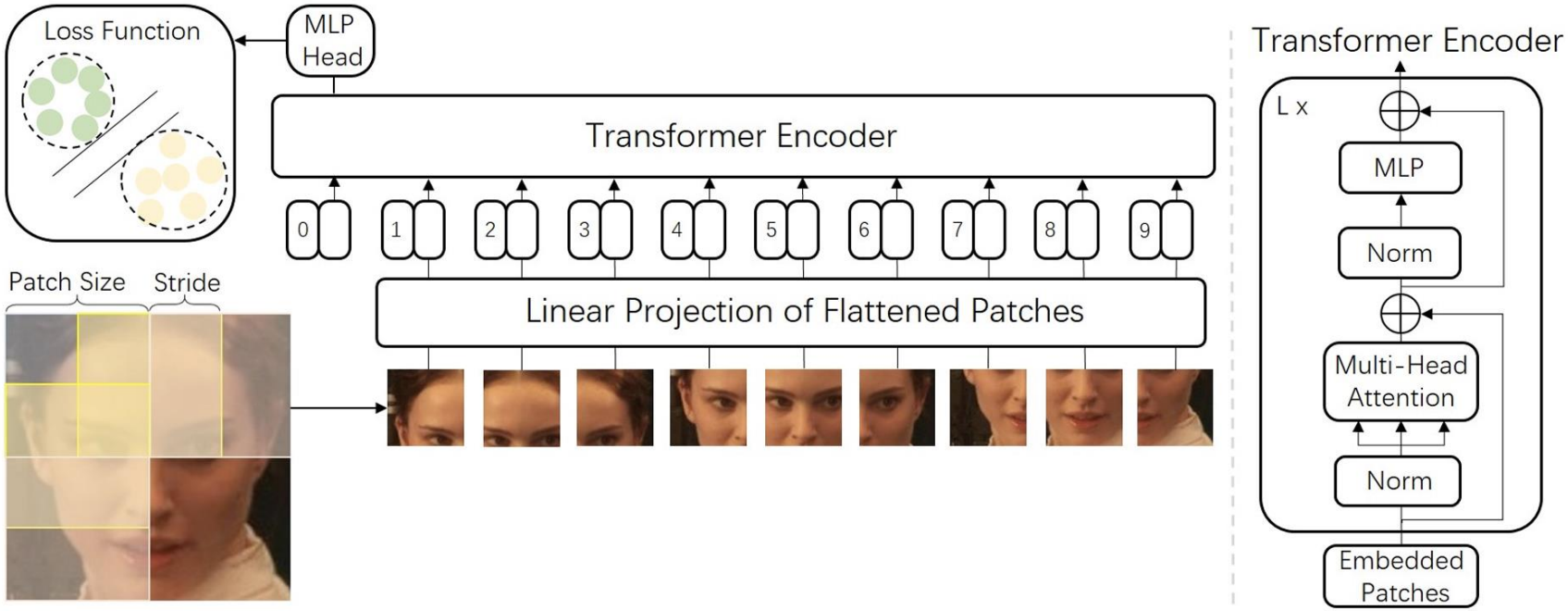


Visual Transformer (ViT)




Class
MLP
Transformer Encoder
Z
z_0 z_1 z_2 ... z_9
PE
PE
Y
y_1 y_2 ... y_9
WX + b
X
x_1 x_2 ... x_9

Visual Transformer (ViT)



Zhong, Y., & Deng, W. (2021). Face transformer for recognition. arXiv preprint arXiv:2103.14803.

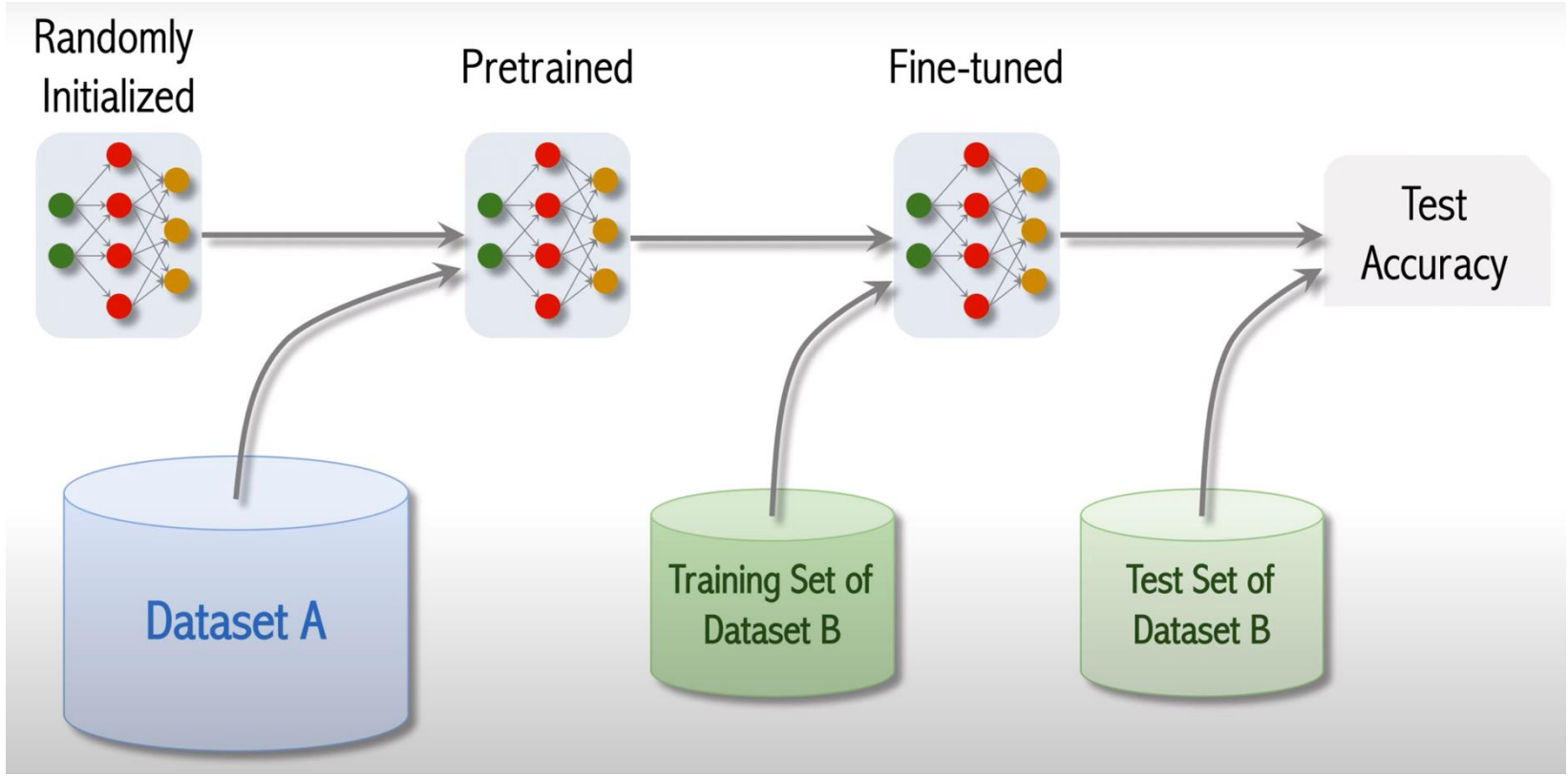
Visual Transformer (ViT)

TABLE II
PERFORMANCE ON LFW [18], SLLFW [19], CALFW [20], CPLFW [21], TALFW [22] CFP-FP [23] AND AGEDB-30 [24] DATABASES.

Training Data	Models	LFW	SLLFW	CALFW	CPLFW	TALFW	CFP-FP	AgeDB-30
CASIA-WebFace	ResNet-100 [12]	99.55	98.65	94.13	90.93	53.17	96.30	95.50
	ViT-P8S8 [1]	97.32	90.78	86.78	80.78	83.05	86.60	81.48
	ViT-P12S8	97.42	90.07	87.35	81.60	84.00	85.56	81.48
MS-Celeb-1M	ResNet-100 [12]	99.82	99.67	96.27	93.43	64.88	96.93	98.27
	ViT-P8S8 [1]	99.83	99.53	95.92	92.55	74.87	96.19	97.82
	T2T-ViT [5]	99.82	99.63	95.85	93.00	71.93	96.59	98.07
	ViT-P10S8	99.77	99.63	95.95	92.93	72.95	96.43	97.83
	ViT-P12S8	99.80	99.55	96.18	93.08	70.13	96.77	98.05

Zhong, Y., & Deng, W. (2021). Face transformer for recognition. arXiv preprint arXiv:2103.14803.

Visual Transformer (ViT)



Shusen Wang: Vision Transformer for Image Classification. https://youtu.be/HZ4j_U3FC94?si=pII9zvAlxx2eIYgq

Visual Transformer (ViT)

	# of Images	# of Classes
ImageNet (Small)	1.3 Million	1 Thousand
ImageNet-21K (Medium)	14 Million	21 Thousand
JFT (Big)	300 Million	18 Thousand

Shusen Wang: Vision Transformer for Image Classification. https://youtu.be/HZ4j_U3FC94?si=pII9zvAlxx2eIYgq

Visual Transformer (ViT): Example



On Skin Lesion Recognition Using Deep Learning: 50 Ways to Choose Your Model

Domingo Mery[1(✉)], Pamela Romero[1], Gabriel Garib[1], Alma Pedro[1], Maria Paz Salinas[2], Javiera Sepulveda[2], Leonel Hidalgo[2], Claudia Prieto[3], and Cristian Navarrete-Dechent[2]

[1] Departments of Computer Science, Pontificia Universidad Catolica de Chile, Santiago, Chile
domingo.mery@uc.cl
[2] Departments of Dermatology, Pontificia Universidad Catolica de Chile, Santiago, Chile
[3] Departments of Electrical Engineering, Pontificia Universidad Catolica de Chile, Santiago, Chile

Abstract. Skin cancer is a highly relevant health problem around the world. The World Health Organization (WHO) reports that one-third of the diagnosed cancers are skin cancer. It is well known that early detection of skin cancer significantly increases the prognosis of patients. In many cases, however, the absence of clinical devices and qualified experts makes this task very difficult. To overcome this problem, in the last years advanced deep learning techniques have been proposed to recognize skin cancer automatically showing promising results. In this work, we evaluate 50 deep learning approaches on the well-known HAM10000 dataset of dermatoscopic images from seven different skin lesion categories. The approaches have been evaluated using the same experimental protocol. In our

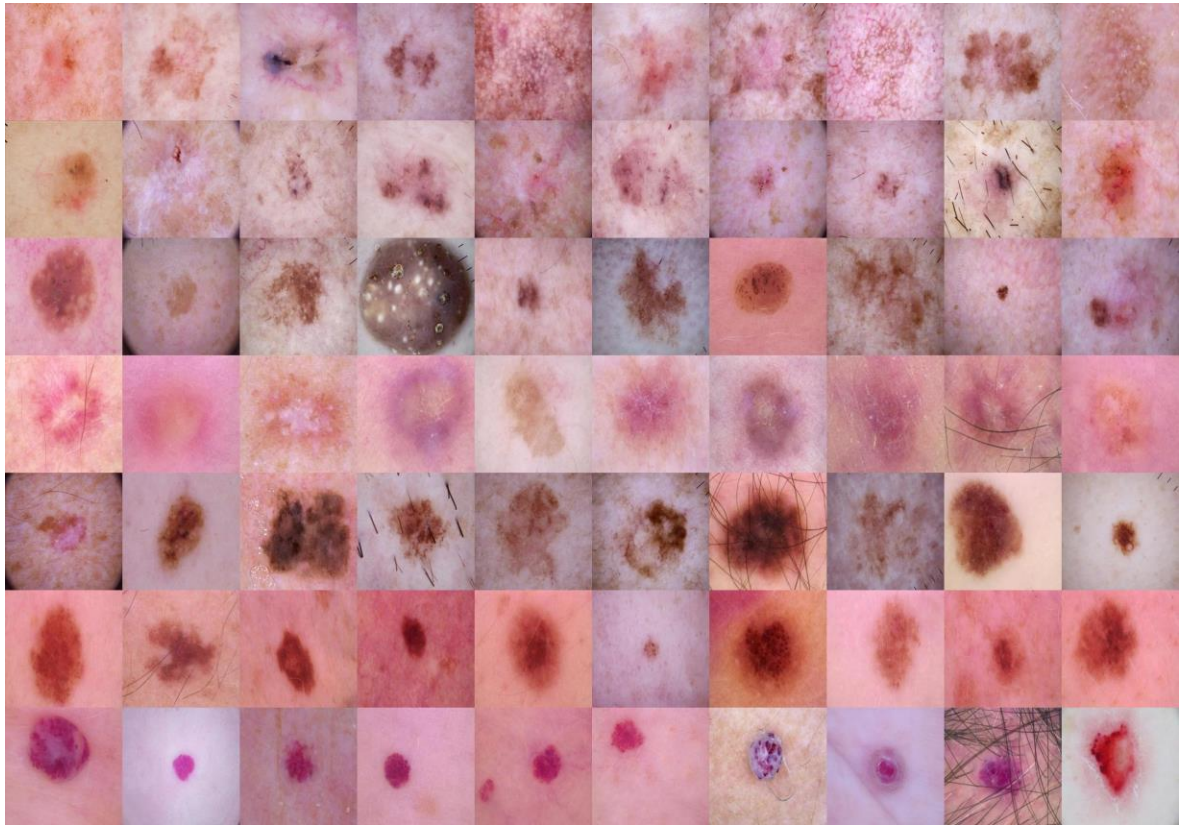
Mery, Domingo, Pamela Romero, Gabriel Garib, Alma Pedro, Maria Paz Salinas, Javiera Sepulveda, Leonel Hidalgo, Claudia Prieto, and Cristian Navarrete-Dechent. "On Skin Lesion Recognition Using Deep Learning: 50 Ways to Choose Your Model." In Pacific-Rim Symposium on Image and Video Technology, pp. 103-116. Cham: Springer International Publishing, 2022.

Evaluated Methods

50 models from
15 different architectures

Database

Class	Acronym	Name
0	akiec	Actinic Keratosis
1	bcc	Basal Cell Carcinoma
2	bkl	Benign Keratosis
3	df	Dermatofibroma
4	mel	Melanoma
5	nv	Melanocytic Nevus
6	vasc	Vascular Lesion



ISIC Dataset – HAM 10000

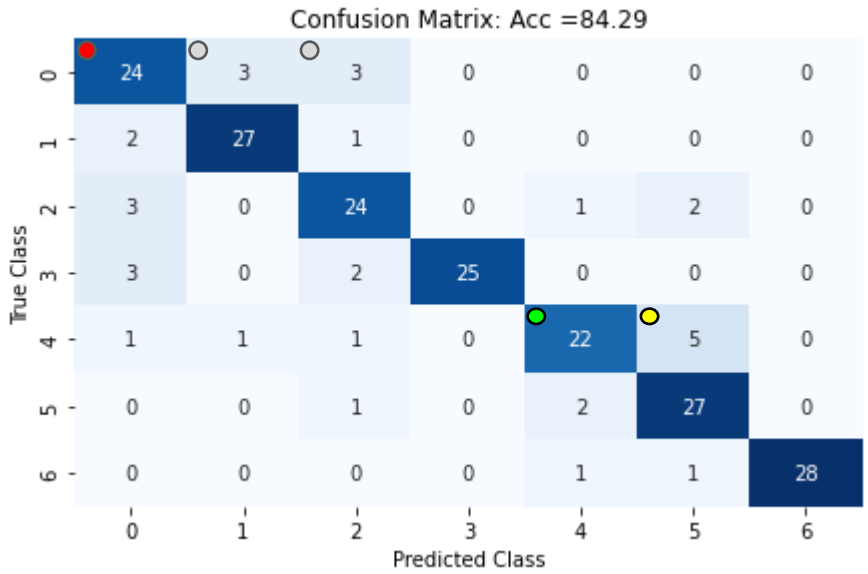
Number of images for training, validation and testing of each class

Subset	class → 0 akiec	1 bcc	2 bkl	3 df	4 mel	5 nv	6 vasc	Total
Training:	210	262	525	70	525	1050	95	2737
Validation:	30	38	75	10	75	150	14	392
Testing:	30	30	30	30	30	30	30	210
Total:	270	330	630	110	630	1230	139	3339

Results
for the
50 models

Year	#	Method
2010	1	ConvNet 5 × 80 [30]
	2	ConvNet 5 × 128 [30]
	3	ConvNet 6S × 224 [30]
	4	ConvNet 6M × 224 [30]
	5	ConvNet 6L × 224 [30]
2014	6	AlexNet [29]
	7	GoogLeNet [47]
	8	VGG11-bn [46]
	9	VGG13-bn [46]
	10	VGG16-bn [46]
	11	VGG19-bn [46]
	12	VGG11 [46]
	13	VGG13 [46]
	14	VGG16 [46]
	15	VGG19 [46]
2015	16	ResNet-18 [22]
	17	ResNet-34 [22]
	18	ResNet-50 [22]
	19	ResNet-101 [22]
	20	ResNet-152 [22]
2016	21	ResNeXt-50 [52]
	22	DenseNet201 [25]
	23	SqueezeNet1-0 [26]
	24	Wide-ResNet-50-2 [54]
2018	25	ShuffleNet-v2-0.5 [34]
	26	ShuffleNet-v2-1.0 [34]
	27	MobileNetv2 [43]
	28	MobileNetv3-S [24]

Confusion Matrix for Best Model (ViT)



Accuracy on testing subset for each architecture

